.NET Full-Stack Developer Interview Handbook

Gated Funnel Structure

This handbook uses a **4-gate progression** model. Each gate corresponds to an experience band and escalates in complexity. Use it as an **interviewer** (evaluate candidates) or as a **candidate** (self-assess and prepare).

Gate	Experience	Question Style	Pass Criteria
Gate 1	Junior (0-3 yr)	Direct conceptual	Can explain core concepts clearly, writes working code
Gate 2	Mid (3-7 yr)	Direct + scenario-based	Applies concepts to real problems, understands trade-offs
Gate 3	Senior (7-15 yr)	Mostly scenario-based	Designs solutions, anticipates failure modes, mentors others
Gate 4	Lead/Architect (15+ yr)	All design/scenario-based	Makes system-level decisions, balances business and technical constraints

How to use the gates: Start every candidate at Gate 1. Advance to the next gate only when the candidate demonstrates consistent competence at the current level. A Senior candidate should still breeze through Gate 1-2 questions; if they stumble, that is a signal.

TABLE OF CONTENTS

PART A — SOLID Principles

- A1. What does the Single Responsibility Principle (SRP) mean? Show a violation and a fix.
- A2. Explain the Open/Closed Principle (OCP) with an example.
- A3. What are the Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), and Dependency Inversion Principle (DIP)? Give a brief code example for each.
- A4. You inherit a 2,000-line service class that handles user registration, email verification, profile management, password resets, and audit logging. How do you refactor it applying SOLID?
- A5. A teammate argues that DIP is unnecessary overhead for a small CRUD app. How do you respond?

PART B — Design Patterns (GoF + Enterprise)

- B1. Explain the Singleton pattern. How do you make it thread-safe in C#?
- B2. What is the difference between Factory Method and Abstract Factory? When would you use each?
- B3. Explain the Strategy pattern. How does it differ from just using an `if/else` or `switch`?
- B4. You need to add audit logging, caching, and retry logic to an existing service without modifying it. Which pattern(s) do you use?
- B5. Describe the Builder pattern. When is it preferable to a constructor with many parameters?

- B6. What is the Repository pattern? How does it differ from just calling the database directly?
- B7. Your team is building a tax document processing pipeline. Documents arrive in different formats (PDF, XML, JSON, CSV). Each format needs parsing, validation, and transformation. How would you design this?
- B8. A colleague uses the Service Locator pattern everywhere instead of constructor injection. What problems does this cause and how do you migrate away from it?
- B9. Design a notification system that supports email, SMS, push notifications, and in-app messages. Different users have different preferences, notifications have priorities, and you need audit logging. Walk through the patterns you would use.
- B10. You discover that your codebase has a God class — a `TaxReturnManager` with 4,000 lines that handles return creation, validation, calculation, e-filing, status tracking, and PDF generation. Outline your decomposition strategy.

PART C — Architecture

- C1. What is the difference between layered architecture and Clean Architecture? When would you choose each?
- C2. Explain Entities, Value Objects, and Aggregates in Domain-Driven Design.
- C3. You are designing a REST API that serves both a web app and mobile clients. The mobile team needs lighter responses, and you need to version the API without breaking existing clients. How do you approach this?
- C4. Explain how you would implement the Mediator pattern (e.g., MediatR) in a .NET project. What are its benefits and when does it become problematic?
- C5. You are tasked with breaking a monolithic .NET API into microservices. What factors drive your decomposition decisions, and what pitfalls do you anticipate?

PART D — Testing

- D1. What are the different types of test doubles (mock, stub, fake, spy)? When do you use each?
- D2. Explain the Test Pyramid. What should you test at each level?
- D3. How do you test async code and controllers in .NET? Show examples with Moq and NUnit or xUnit.
- D4. A test suite has 500 tests that take 15 minutes to run. Many tests are flaky. Your team avoids writing new tests because the suite is unreliable. How do you rehabilitate it?
- D5. Describe TDD (Red-Green-Refactor). Walk through developing a tax bracket calculator using TDD.

PART E — Cloud & DevOps

- E1. Compare Azure App Service, Azure Functions, and containerized deployments (AKS/ACA). When would you choose each for a .NET API?
- E2. Explain the key differences between Azure Service Bus and AWS SQS/SNS. When would you use a message queue vs a pub/sub topic?
- E3. Write a Dockerfile for a .NET 8 API with multi-stage build. Explain each stage and why multi-stage matters.
- E4. Describe blue-green and canary deployment strategies. How would you implement canary for a .NET API on Kubernetes?
- E5. What are the 12-factor app principles? Which ones are most relevant to a .NET cloud deployment?

PART F — Git & Version Control

- F1. Explain the difference between `git merge` and `git rebase`. When would you use each?
- F2. Describe GitFlow, trunk-based development, and GitHub Flow. Which do you prefer and why?
- F3. You are reviewing a PR and notice the branch has 47 commits including "WIP", "fix typo", "oops", and "actually fix it this time." How do you handle this as a reviewer? What Git techniques can clean this up?
- F4. Two developers working on the same file create conflicting changes. Walk through how you resolve a merge conflict, and how to minimize conflicts in the first place.

PART G — Data Structures & Algorithms (Essentials)

- G1. Explain Big-O notation. What are the time complexities of common operations on .NET collections?
- G2. When would you use a Dictionary vs a List vs a HashSet in C#? Give scenarios for each.
- G3. You have a list of 1 million tax returns and need to find all returns for a specific taxpayer. The operation is performed frequently. How do you optimize this?
- G4. Explain the difference between IEnumerable, ICollection, and IList in .NET. When should you expose each as a return type or parameter?

PART H — Agile & Process

- H1. What is the difference between Scrum and Kanban? When would you prefer one over the other?
- H2. What is "Definition of Done" and why does it matter?
- H3. Your team has accumulated significant technical debt. The product owner resists dedicating sprint capacity to debt reduction because "there are no new features." How do you make the case?
- H4. How do you estimate work effectively? What are story points and when do they fail?

Appendices

- Appendix A — Quick Reference
- Appendix B — Gate Assessment Summary
- Appendix C — Question Index

PART A — SOLID Principles

Gate 1 — Conceptual Understanding

A1. What does the Single Responsibility Principle (SRP) mean? Show a violation and a fix.

Expected Answer:

A class should have only one reason to change — it should do one thing and do it well.

Violation — class handles both order logic and persistence:

```
public class OrderService
{
    public decimal CalculateTotal(Order order)
    {
```

```
        decimal total = 0;
        foreach (var item in order.Items)
            total += item.Price * item.Quantity;
        return total;
    }

    public void SaveToDatabase(Order order)
    {
        using var connection = new SqlConnection("...");
        connection.Open();
        // INSERT INTO Orders ...
    }

    public void SendConfirmationEmail(Order order)
    {
        var smtp = new SmtpClient();
        // send email...
    }
}
```

Fix — separate responsibilities:

```
public class OrderCalculator
{
    public decimal CalculateTotal(Order order)
    {
        return order.Items.Sum(i => i.Price * i.Quantity);
    }
}

public class OrderRepository
{
    private readonly IDbConnection _connection;
    public OrderRepository(IDbConnection connection) => _connection = connection;

    public void Save(Order order)
    {
        // INSERT INTO Orders ...
    }
}

public class OrderNotificationService
{
    private readonly IEmailSender _emailSender;
    public OrderNotificationService(IEmailSender emailSender) => _emailSender =
emailSender;

    public void SendConfirmation(Order order)
    {
        _emailSender.Send(order.CustomerEmail, "Order Confirmed", "...");
    }
}
```

What to listen for: The candidate articulates "one reason to change" rather than just "one thing." They can identify concrete change vectors (database technology, email provider, business rules).

A2. Explain the Open/Closed Principle (OCP) with an example.

Expected Answer:

Software entities should be open for extension but closed for modification. You should be able to add new behavior without changing existing code.

Violation — modifying existing code to add new discount types:

```
public class DiscountCalculator
{
    public decimal Calculate(Order order, string discountType)
    {
        switch (discountType)
        {
            case "Percentage":
                return order.Total * 0.1m;
            case "FixedAmount":
                return 10m;
            case "BuyOneGetOne": // Added later — modifying existing class
                return order.Items.Min(i => i.Price);
            default:
                return 0;
        }
    }
}
```

Fix — extend through abstraction:

```
public interface IDiscountStrategy
{
    decimal Calculate(Order order);
}

public class PercentageDiscount : IDiscountStrategy
{
    private readonly decimal _percentage;
    public PercentageDiscount(decimal percentage) => _percentage = percentage;
    public decimal Calculate(Order order) => order.Total * _percentage;
}

public class FixedAmountDiscount : IDiscountStrategy
{
    private readonly decimal _amount;
```

```
    public FixedAmountDiscount(decimal amount) => _amount = amount;
    public decimal Calculate(Order order) => Math.Min(_amount, order.Total);
}

// Adding new discount — no existing code modified
public class BuyOneGetOneDiscount : IDiscountStrategy
{
    public decimal Calculate(Order order) => order.Items.Min(i => i.Price);
}

public class DiscountCalculator
{
    public decimal Calculate(Order order, IDiscountStrategy strategy)
    {
        return strategy.Calculate(order);
    }
}
```

What to listen for: Understands that "closed for modification" does not mean "never touch the code again" — it means the core logic does not need to change when behavior is extended. Mentions polymorphism or interfaces as the mechanism.

A3. What are the Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), and Dependency Inversion Principle (DIP)? Give a brief code example for each.

Expected Answer:

LSP — Subtypes must be substitutable for their base types without breaking correctness.

Violation: `Square` inheriting `Rectangle` where setting Width independently breaks the invariant.

```
// Violation
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
    public int Area() => Width * Height;
}

public class Square : Rectangle
{
    public override int Width
    {
        set { base.Width = value; base.Height = value; }
    }
    public override int Height
    {
        set { base.Width = value; base.Height = value; }
    }
}
```

```
// This breaks — caller expects Width and Height to be independent
void Resize(Rectangle r)
{
    r.Width = 5;
    r.Height = 10;
    Debug.Assert(r.Area() == 50); // Fails for Square (100)
}
```

Fix: Use a common `IShape` interface instead of inheritance.

ISP — Clients should not be forced to depend on methods they do not use.

```
// Violation — forces read-only clients to implement write methods
public interface IRepository<T>
{
    T GetById(int id);
    IEnumerable<T> GetAll();
    void Add(T entity);
    void Update(T entity);
    void Delete(int id);
}

// Fix — split interfaces by usage
public interface IReadRepository<T>
{
    T GetById(int id);
    IEnumerable<T> GetAll();
}

public interface IWriteRepository<T>
{
    void Add(T entity);
    void Update(T entity);
    void Delete(int id);
}

public interface IRepository<T> : IReadRepository<T>, IWriteRepository<T> { }
```

DIP — High-level modules should not depend on low-level modules. Both should depend on abstractions.

```
// Violation — high-level OrderService depends on concrete SqlOrderRepository
public class OrderService
{
    private readonly SqlOrderRepository _repo = new SqlOrderRepository();
}

// Fix — depend on abstraction
```

```
public class OrderService
{
    private readonly IOrderRepository _repo;
    public OrderService(IOrderRepository repo) => _repo = repo;
}
```

What to listen for: For LSP, the candidate talks about behavioral contracts, not just type compatibility. For ISP, they mention real scenarios where fat interfaces cause pain (e.g., implementing no-op methods). For DIP, they connect it to dependency injection but clarify that DIP is the principle and DI is a technique.

Gate 2 — Scenario-Based

A4. You inherit a 2,000-line service class that handles user registration, email verification, profile management, password resets, and audit logging. How do you refactor it applying SOLID?

Expected Answer:

1. **Identify responsibilities** — list the distinct change vectors: registration rules, email sending infrastructure, profile CRUD, password security policy, audit storage.
2. **Extract interfaces first** — define `IRegistrationService`, `IEmailVerificationService`, `IProfileService`, `IPasswordResetService`, `IAuditLogger`.
3. **Move methods** — each extracted class gets the methods and private helpers related to its concern.
4. **Wire through DI** — register each new service; the original class becomes a thin facade or is deleted.
5. **Apply OCP** — if registration has multiple paths (social, email, SSO), use a strategy pattern rather than switch statements.
6. **Test incrementally** — extract one responsibility at a time; run existing tests after each extraction; add new unit tests for each extracted service.

What to listen for: Methodical approach rather than "rewrite everything." Mentions preserving behavior (refactoring, not rewriting). Talks about how to handle the transition period where both old and new code exist.

Red flag: Wants to rewrite from scratch without preserving existing tests. Cannot articulate a step-by-step extraction sequence. Says "just split it into smaller classes" without explaining the criteria for splitting.

A5. A teammate argues that DIP is unnecessary overhead for a small CRUD app. How do you respond?

Expected Answer:

They have a point for truly throwaway prototypes. But even small apps benefit from DIP for testability — you cannot unit test a service that `new`s up its dependencies. The overhead is minimal with modern DI containers (one line of registration per service). The real cost is *not* applying DIP: when the "small CRUD app" grows (and it always does), retrofitting abstractions is far more expensive than starting with them. Compromise: apply DIP at the service layer boundaries (controllers depend on service interfaces, services depend on repository interfaces) but skip it for internal helpers within a service.

What to listen for: Nuance — not "always apply everywhere" or "never needed." Mentions testability as the primary pragmatic benefit. Acknowledges legitimate cases where the overhead is not worthwhile.

Red flag: Dogmatic "always apply SOLID everywhere" without acknowledging trade-offs. Or conversely, "SOLID is academic theory, real code does not need it."

↑ Back to Index

PART B — Design Patterns (GoF + Enterprise)

Gate 1 — Conceptual

B1. Explain the Singleton pattern. How do you make it thread-safe in C#?

Expected Answer:

Singleton ensures a class has only one instance and provides a global access point to it.

Naive (not thread-safe):

```
public class ConnectionPool
{
    private static ConnectionPool _instance;

    private ConnectionPool() { }

    public static ConnectionPool Instance
    {
        get
        {
            if (_instance == null)
                _instance = new ConnectionPool(); // Race condition
            return _instance;
        }
    }
}
```

Thread-safe options:

```
// Option 1: Lazy<T> (recommended)
public class ConnectionPool
{
    private static readonly Lazy<ConnectionPool> _instance =
        new Lazy<ConnectionPool>(() => new ConnectionPool());

    private ConnectionPool() { }

    public static ConnectionPool Instance => _instance.Value;
```

```
}

// Option 2: Static initializer (eager, simple)
public class ConnectionPool
{
    private static readonly ConnectionPool _instance = new ConnectionPool();

    private ConnectionPool() { }

    public static ConnectionPool Instance => _instance;
}

// Option 3: Double-check locking (manual, rarely needed)
public class ConnectionPool
{
    private static volatile ConnectionPool _instance;
    private static readonly object _lock = new object();

    private ConnectionPool() { }

    public static ConnectionPool Instance
    {
        get
        {
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                        _instance = new ConnectionPool();
                }
            }
            return _instance;
        }
    }
}
```

What to listen for: Knows that `Lazy<T>` is the idiomatic C# approach. Understands *why* the naive version fails (two threads can both see `null` and create separate instances). Mentions that in modern .NET, you almost always use DI container `AddSingleton<T>()` instead of the classic pattern.

B2. What is the difference between Factory Method and Abstract Factory? When would you use each?

Expected Answer:

Factory Method — a single method (often virtual) that subclasses override to create different products.

```
public abstract class NotificationSender
{
```

```
    // Factory method
    protected abstract INotification CreateNotification(string message);

    public void Send(string message)
    {
        var notification = CreateNotification(message);
        notification.Deliver();
    }
}

public class EmailNotificationSender : NotificationSender
{
    protected override INotification CreateNotification(string message)
        => new EmailNotification(message);
}

public class SmsNotificationSender : NotificationSender
{
    protected override INotification CreateNotification(string message)
        => new SmsNotification(message);
}
```

Abstract Factory — creates families of related objects without specifying concrete classes. Use when you need to create multiple related objects that must be compatible.

```
public interface IUIFactory
{
    IButton CreateButton();
    ITextBox CreateTextBox();
    IDropdown CreateDropdown();
}

public class MaterialUIFactory : IUIFactory
{
    public IButton CreateButton() => new MaterialButton();
    public ITextBox CreateTextBox() => new MaterialTextBox();
    public IDropdown CreateDropdown() => new MaterialDropdown();
}

public class BootstrapUIFactory : IUIFactory
{
    public IButton CreateButton() => new BootstrapButton();
    public ITextBox CreateTextBox() => new BootstrapTextBox();
    public IDropdown CreateDropdown() => new BootstrapDropdown();
}
```

When to use which:

- Factory Method: Single product, decision is which *variant* to create.

- Abstract Factory: Family of products that must be used together (mixing Material button with Bootstrap dropdown would be inconsistent).

What to listen for: Clear distinction between "one product, multiple variants" (Factory Method) and "multiple products, multiple families" (Abstract Factory). Real-world examples beyond textbook shapes/animals.

B3. Explain the Strategy pattern. How does it differ from just using an `if/else` or `switch`?

Expected Answer:

Strategy encapsulates interchangeable algorithms behind a common interface. The client delegates to the strategy without knowing the implementation.

```
public interface ITaxCalculationStrategy
{
    decimal CalculateTax(decimal income, string state);
}

public class FederalTaxStrategy : ITaxCalculationStrategy
{
    public decimal CalculateTax(decimal income, string state)
    {
        // Federal bracket calculation
        if (income <= 10_275m) return income * 0.10m;
        if (income <= 41_775m) return 1_027.50m + (income - 10_275m) * 0.12m;
        // ... more brackets
        return 0;
    }
}

public class StateTaxStrategy : ITaxCalculationStrategy
{
    public decimal CalculateTax(decimal income, string state)
    {
        return state switch
        {
            "CA" => income * 0.0725m,
            "TX" => 0m,  // No state income tax
            "NY" => income * 0.0685m,
            _ => income * 0.05m
        };
    }
}

public class TaxService
{
    private readonly ITaxCalculationStrategy _strategy;

    public TaxService(ITaxCalculationStrategy strategy)
        => _strategy = strategy;
```

```
    public decimal ComputeTax(decimal income, string state)
        => _strategy.CalculateTax(income, state);
}
```

Difference from `if/else`:

- `if/else` requires modifying the existing class to add new behavior (violates OCP).
- Strategy allows adding new algorithms without touching existing code.
- Strategies are independently testable.
- Strategies can be swapped at runtime via DI or configuration.

What to listen for: Connects Strategy to OCP. Mentions DI as the usual wiring mechanism in .NET. Understands that for 2-3 simple cases a switch might be fine — Strategy shines when algorithms are complex or change frequently.

Gate 2 — Applied

B4. You need to add audit logging, caching, and retry logic to an existing service without modifying it. Which pattern(s) do you use?

Expected Answer:

Decorator pattern — wrap the service with each cross-cutting concern as a separate decorator, each implementing the same interface.

```
public interface IOrderService
{
    Task<Order> GetOrderAsync(int id);
    Task<int> CreateOrderAsync(Order order);
}

// Original implementation
public class OrderService : IOrderService
{
    public async Task<Order> GetOrderAsync(int id) { /* ... */ }
    public async Task<int> CreateOrderAsync(Order order) { /* ... */ }
}

// Caching decorator
public class CachingOrderService : IOrderService
{
    private readonly IOrderService _inner;
    private readonly IMemoryCache _cache;

    public CachingOrderService(IOrderService inner, IMemoryCache cache)
    {
        _inner = inner;
        _cache = cache;
    }
```

```
    public async Task<Order> GetOrderAsync(int id)
    {
        return await _cache.GetOrCreateAsync($"order:{id}", async entry =>
        {
            entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
            return await _inner.GetOrderAsync(id);
        });
    }

    public async Task<int> CreateOrderAsync(Order order)
    {
        var id = await _inner.CreateOrderAsync(order);
        _cache.Remove($"order:{id}");
        return id;
    }
}

// Retry decorator
public class RetryOrderService : IOrderService
{
    private readonly IOrderService _inner;
    private readonly int _maxRetries;

    public RetryOrderService(IOrderService inner, int maxRetries = 3)
    {
        _inner = inner;
        _maxRetries = maxRetries;
    }

    public async Task<Order> GetOrderAsync(int id)
    {
        for (int attempt = 0; ; attempt++)
        {
            try { return await _inner.GetOrderAsync(id); }
            catch when (attempt < _maxRetries)
            {
                await Task.Delay(TimeSpan.FromMilliseconds(100 * Math.Pow(2,
attempt)));
            }
        }
    }

    public async Task<int> CreateOrderAsync(Order order)
    {
        // Similar retry logic
        return await _inner.CreateOrderAsync(order);
    }
}

// Audit decorator
public class AuditOrderService : IOrderService
{
    private readonly IOrderService _inner;
    private readonly IAuditLogger _audit;
```

```
    public AuditOrderService(IOrderService inner, IAuditLogger audit)
    {
        _inner = inner;
        _audit = audit;
    }

    public async Task<Order> GetOrderAsync(int id)
    {
        var result = await _inner.GetOrderAsync(id);
        await _audit.LogAsync("GetOrder", new { id });
        return result;
    }

    public async Task<int> CreateOrderAsync(Order order)
    {
        var id = await _inner.CreateOrderAsync(order);
        await _audit.LogAsync("CreateOrder", new { orderId = id });
        return id;
    }
}

// DI registration — decorators compose from inside out
services.AddScoped<OrderService>();
services.AddScoped<IOrderService>(sp =>
    new AuditOrderService(
        new RetryOrderService(
            new CachingOrderService(
                sp.GetRequiredService<OrderService>(),
                sp.GetRequiredService<IMemoryCache>()),
            maxRetries: 3),
        sp.GetRequiredService<IAuditLogger>()));
```

What to listen for: Names the Decorator pattern explicitly. Understands the composition order matters (retry should wrap caching, not the other way around). Mentions alternatives like middleware pipeline or MediatR behaviors for cross-cutting concerns. Acknowledges the boilerplate cost and when Scrutor or similar libraries help.

Red flag: Suggests modifying the original service directly. Confuses Decorator with Proxy. Cannot explain the DI registration for nested decorators.

B5. Describe the Builder pattern. When is it preferable to a constructor with many parameters?

Expected Answer:

Builder separates the construction of a complex object from its representation. It is preferable when:

- The constructor would need 5+ parameters
- Some parameters are optional
- The construction process involves multiple steps or validation

```
public class TaxReturn
{
    public string FilingStatus { get; }
    public int TaxYear { get; }
    public decimal GrossIncome { get; }
    public decimal Deductions { get; }
    public List<W2Form> W2s { get; }
    public List<Form1099> Form1099s { get; }
    public Address MailingAddress { get; }
    public bool IsAmended { get; }

    private TaxReturn(Builder builder)
    {
        FilingStatus = builder.FilingStatus;
        TaxYear = builder.TaxYear;
        GrossIncome = builder.GrossIncome;
        Deductions = builder.Deductions;
        W2s = builder.W2s;
        Form1099s = builder.Form1099s;
        MailingAddress = builder.MailingAddress;
        IsAmended = builder.IsAmended;
    }

    public class Builder
    {
        // Required
        public string FilingStatus { get; }
        public int TaxYear { get; }

        // Optional with defaults
        public decimal GrossIncome { get; private set; }
        public decimal Deductions { get; private set; }
        public List<W2Form> W2s { get; private set; } = new();
        public List<Form1099> Form1099s { get; private set; } = new();
        public Address MailingAddress { get; private set; }
        public bool IsAmended { get; private set; }

        public Builder(string filingStatus, int taxYear)
        {
            FilingStatus = filingStatus;
            TaxYear = taxYear;
        }

        public Builder WithIncome(decimal grossIncome)
        {
            GrossIncome = grossIncome;
            return this;
        }

        public Builder WithDeductions(decimal deductions)
        {
            Deductions = deductions;
            return this;
```

```
        }

        public Builder AddW2(W2Form w2)
        {
            W2s.Add(w2);
            return this;
        }

        public Builder AddForm1099(Form1099 form)
        {
            Form1099s.Add(form);
            return this;
        }

        public Builder WithMailingAddress(Address address)
        {
            MailingAddress = address;
            return this;
        }

        public Builder AsAmended()
        {
            IsAmended = true;
            return this;
        }

        public TaxReturn Build()
        {
            // Validate required state
            if (string.IsNullOrEmpty(FilingStatus))
                throw new InvalidOperationException("Filing status is required.");
            return new TaxReturn(this);
        }
    }
}

// Usage
var taxReturn = new TaxReturn.Builder("Single", 2025)
    .WithIncome(85_000m)
    .WithDeductions(13_850m)
    .AddW2(new W2Form { EmployerEin = "12-3456789", Wages = 85_000m })
    .WithMailingAddress(new Address { Street = "123 Main St", City = "Austin",
State = "TX" })
    .Build();
```

What to listen for: Distinguishes required vs optional parameters in the builder. Mentions validation in the `Build()` method. Knows that C# records with `with` expressions and init-only properties reduce the need for builders in simple cases. Mentions fluent API readability benefit.

B6. What is the Repository pattern? How does it differ from just calling the database directly?

Expected Answer:

Repository mediates between the domain and data mapping layers, acting as an in-memory collection of domain objects. It abstracts the data access technology.

```
public interface ITaxReturnRepository
{
    Task<TaxReturn> GetByIdAsync(int id);
    Task<IEnumerable<TaxReturn>> GetByTaxpayerAsync(string taxpayerId, int
taxYear);
    Task<int> AddAsync(TaxReturn taxReturn);
    Task UpdateAsync(TaxReturn taxReturn);
}

public class TaxReturnRepository : ITaxReturnRepository
{
    private readonly IDatabase _db;

    public TaxReturnRepository(IDatabase db) => _db = db;

    public async Task<TaxReturn> GetByIdAsync(int id)
    {
        return await _db.SingleOrDefaultAsync<TaxReturn>(
            "SELECT * FROM TaxReturns WHERE Id = @0", id);
    }

    public async Task<IEnumerable<TaxReturn>> GetByTaxpayerAsync(
        string taxpayerId, int taxYear)
    {
        return await _db.FetchAsync<TaxReturn>(
            "WHERE TaxpayerId = @0 AND TaxYear = @1", taxpayerId, taxYear);
    }

    public async Task<int> AddAsync(TaxReturn taxReturn)
    {
        await _db.InsertAsync(taxReturn);
        return taxReturn.Id;
    }

    public async Task UpdateAsync(TaxReturn taxReturn)
    {
        await _db.UpdateAsync(taxReturn);
    }
}
```

Benefits over direct database calls:

- Testability: Mock `ITaxReturnRepository` without a database.
- Encapsulation: SQL lives in one place, not scattered across services.
- Swappability: Change from SQL Server to PostgreSQL by replacing the implementation.
- Query reuse: Common queries are defined once.

What to listen for: Understands that Repository is about abstraction, not just "a class with CRUD methods." Mentions testability as the primary benefit. Knows the pattern can be over-applied (generic `IRepository<T>` for everything is an anti-pattern when queries are domain-specific).

Gate 3 — Scenario-Based

B7. Your team is building a tax document processing pipeline. Documents arrive in different formats (PDF, XML, JSON, CSV). Each format needs parsing, validation, and transformation. How would you design this?

Expected Answer:

Combine **Template Method** for the common pipeline structure with **Strategy** (or **Chain of Responsibility**) for format-specific parsing.

```
// Template Method defines the pipeline skeleton
public abstract class DocumentProcessor
{
    public async Task<ProcessingResult> ProcessAsync(Stream document)
    {
        var rawData = await ParseAsync(document);       // Abstract — varies by
format
        var validated = Validate(rawData);               // Common validation
logic
        var transformed = Transform(validated);          // Abstract — varies by
output
        await StoreAsync(transformed);                   // Common storage logic
        return new ProcessingResult { Success = true, DocumentId = transformed.Id
};
    }

    protected abstract Task<RawDocumentData> ParseAsync(Stream document);
    protected abstract TransformedDocument Transform(ValidatedData data);

    protected virtual ValidatedData Validate(RawDocumentData data)
    {
        // Common validation — subclasses can override for format-specific rules
        if (string.IsNullOrEmpty(data.TaxpayerId))
            throw new ValidationException("Taxpayer ID is required.");
        return new ValidatedData(data);
    }

    protected async Task StoreAsync(TransformedDocument doc)
    {
        // Common persistence logic
    }
}

// Format-specific processors
public class PdfDocumentProcessor : DocumentProcessor
{
```

```
    protected override async Task<RawDocumentData> ParseAsync(Stream document)
    {
        // PDF-specific parsing with iTextSharp or similar
    }

    protected override TransformedDocument Transform(ValidatedData data)
    {
        // PDF-specific transformation
    }
}

public class XmlDocumentProcessor : DocumentProcessor
{
    protected override async Task<RawDocumentData> ParseAsync(Stream document)
    {
        // XML deserialization
    }

    protected override TransformedDocument Transform(ValidatedData data)
    {
        // XML-specific mapping
    }
}

// Factory to select the right processor
public class DocumentProcessorFactory
{
    private readonly IServiceProvider _serviceProvider;

    public DocumentProcessor Create(string contentType)
    {
        return contentType switch
        {
            "application/pdf" =>
_serviceProvider.GetRequiredService<PdfDocumentProcessor>(),
            "application/xml" =>
_serviceProvider.GetRequiredService<XmlDocumentProcessor>(),
            "application/json" =>
_serviceProvider.GetRequiredService<JsonDocumentProcessor>(),
            "text/csv" =>
_serviceProvider.GetRequiredService<CsvDocumentProcessor>(),
            _ => throw new NotSupportedException($"Format '{contentType}' is not
supported.")
        };
    }
}
```

What to listen for: Combines patterns rather than forcing one. Identifies the invariant parts (validate, store) vs variant parts (parse, transform). Discusses extensibility — adding a new format requires only a new subclass and a factory mapping. Mentions error handling strategy for the pipeline (fail fast vs collect errors).

Red flag: Giant switch statement in a single class. No mention of extensibility. Over-engineers with every GoF pattern simultaneously.

B8. A colleague uses the Service Locator pattern everywhere instead of constructor injection. What problems does this cause and how do you migrate away from it?

Expected Answer:

Problems with Service Locator:

```
// Service Locator — anti-pattern
public class OrderService
{
    public void ProcessOrder(Order order)
    {
        // Hidden dependencies — not visible from constructor
        var repo = ServiceLocator.Resolve<IOrderRepository>();
        var logger = ServiceLocator.Resolve<ILogger>();
        var emailSender = ServiceLocator.Resolve<IEmailSender>();

        repo.Save(order);
        emailSender.Send(order.CustomerEmail, "Confirmed");
        logger.LogInformation("Order processed: {OrderId}", order.Id);
    }
}
```

1. **Hidden dependencies** — You cannot tell what a class needs by looking at its constructor. Discovery requires reading every method.
2. **Hard to test** — Must configure a static service locator in tests; cannot simply pass mocks through the constructor.
3. **Runtime failures** — Missing registrations are discovered at runtime, not at startup.
4. **Coupling** — Every class depends on the ServiceLocator itself, a God object.

Migration approach:

1. Add constructor parameters for each resolved dependency.
2. Have the Service Locator call sites pass dependencies through constructors.
3. Register services in the DI container.
4. Remove Service Locator calls one class at a time, starting from leaf dependencies.
5. Delete the Service Locator class when no references remain.

```
// After migration — constructor injection
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly ILogger _logger;
    private readonly IEmailSender _emailSender;
```

```
    public OrderService(
        IOrderRepository repo,
        ILogger logger,
        IEmailSender emailSender)
    {
        _repo = repo;
        _logger = logger;
        _emailSender = emailSender;
    }

    public void ProcessOrder(Order order)
    {
        _repo.Save(order);
        _emailSender.Send(order.CustomerEmail, "Confirmed");
        _logger.LogInformation("Order processed: {OrderId}", order.Id);
    }
}
```

What to listen for: Identifies "hidden dependencies" as the core problem. Describes incremental migration (not big-bang rewrite). Mentions that Service Locator is sometimes acceptable for framework code where constructor injection is impossible (e.g., middleware factories, attribute-based injection).

Red flag: Cannot articulate why Service Locator is problematic. Says "just rewrite everything at once." Does not mention testability.

Gate 4 — Design/Scenario-Based

B9. Design a notification system that supports email, SMS, push notifications, and in-app messages. Different users have different preferences, notifications have priorities, and you need audit logging. Walk through the patterns you would use.

Expected Answer:

This requires composing several patterns:

1. **Strategy** — `INotificationChannel` interface with implementations for each delivery mechanism (Email, SMS, Push, InApp).
2. **Observer/Pub-Sub** — Events trigger notifications; a `NotificationDispatcher` observes domain events and routes to the appropriate channels.
3. **Chain of Responsibility** — Priority-based filtering: urgent notifications bypass user preferences; low-priority notifications respect quiet hours.
4. **Decorator** — Wrap channels with audit logging and retry decorators.
5. **Builder** — Construct notification payloads that vary by channel (email has subject + HTML body, SMS has character limits, push has payload constraints).

```
Domain Event
     |
     v
NotificationDispatcher (Observer)
```

```
      |
      v
 PriorityFilter (Chain of Responsibility)
      |
      v
 UserPreferenceResolver -- determines which channels
      |
      v
 INotificationChannel[] (Strategy)
   |        |        |         |
 Email    SMS     Push      InApp
   |        |        |         |
 [AuditDecorator wraps each channel] (Decorator)
```

```
public interface INotificationChannel
{
    string ChannelType { get; }
    Task SendAsync(NotificationPayload payload);
    bool CanHandle(NotificationPriority priority);
}

public class NotificationDispatcher
{
    private readonly IEnumerable<INotificationChannel> _channels;
    private readonly IUserPreferenceService _preferences;
    private readonly IPriorityFilter _priorityFilter;

    public NotificationDispatcher(
        IEnumerable<INotificationChannel> channels,
        IUserPreferenceService preferences,
        IPriorityFilter priorityFilter)
    {
        _channels = channels;
        _preferences = preferences;
        _priorityFilter = priorityFilter;
    }

    public async Task DispatchAsync(string userId, DomainEvent domainEvent)
    {
        var notification = MapToNotification(domainEvent);

        if (!_priorityFilter.ShouldSend(notification))
            return;

        var prefs = await _preferences.GetPreferencesAsync(userId);
        var enabledChannels = _channels
            .Where(c => prefs.EnabledChannels.Contains(c.ChannelType))
            .Where(c => c.CanHandle(notification.Priority));

        var tasks = enabledChannels.Select(c =>
c.SendAsync(notification.Payload));
```

```
            await Task.WhenAll(tasks);
        }
    }
```

What to listen for: Does not force a single pattern — composes multiple patterns naturally. Considers failure modes (what if SMS provider is down? queue for retry). Discusses scalability (async message queue for high volume). Mentions user preference storage and how priority overrides preferences for critical notifications.

Red flag: Over-designs with unnecessary abstractions. Under-designs by putting everything in one class. Does not consider failure handling or scalability. Cannot explain why each pattern was chosen.

B10. You discover that your codebase has a God class — a `TaxReturnManager` with 4,000 lines that handles return creation, validation, calculation, e-filing, status tracking, and PDF generation. Outline your decomposition strategy.

Expected Answer:

1. **Map dependencies** — Identify which methods call which private fields and helpers. Group methods by the data they touch (the "responsibility clusters").
2. **Identify seams** — Find natural boundaries: creation/validation vs calculation vs e-filing vs PDF generation. These are different bounded contexts.
3. **Extract bottom-up starting with leaf services:**
   - `TaxCalculationService` — pure computation, no I/O, easiest to test
   - `PdfGenerationService` — isolated I/O concern, clear input/output
   - `EFileSubmissionService` — external API integration, needs its own retry/error handling
   - `ReturnStatusTracker` — state machine, can be modeled explicitly
   - `ReturnValidationService` — rule engine, benefits from Specification pattern
4. **Introduce facades if needed** — If callers depend on the original class API, keep a thin `TaxReturnFacade` that delegates to the extracted services. Remove the facade once callers are migrated.
5. **Strangler Fig migration** — Do not extract everything at once. Extract one service, redirect calls to it, verify in production, repeat.
6. **Tests as safety net** — Before extracting, ensure the God class has integration tests covering the major paths. After extraction, add unit tests for each new service.

What to listen for: Talks about dependency mapping before cutting code. Uses incremental extraction (Strangler Fig). Considers backward compatibility for callers. Identifies which piece to extract first based on risk and testability. Mentions that the God class likely violates every SOLID principle.

Red flag: Proposes a big-bang rewrite. No mention of testing strategy during migration. Cannot identify natural seam boundaries. Proposes creating equally large "Manager" classes.

↑ Back to Index

PART C — Architecture

Gate 1 — Conceptual

C1. What is the difference between layered architecture and Clean Architecture? When would you choose each?

Expected Answer:

Layered Architecture — horizontal slices with a strict dependency direction: Presentation -> Business -> Data Access.

```
┌──────────────────────────┐
│    Presentation Layer    │  (Controllers, Views)
├──────────────────────────┤
│    Business Logic Layer  │  (Services, Rules)
├──────────────────────────┤
│    Data Access Layer     │  (Repositories, ORM)
├──────────────────────────┤
│       Database           │
└──────────────────────────┘
      Dependencies flow DOWN
```

Clean Architecture (Onion/Hexagonal) — concentric rings where the domain is at the center and depends on nothing. Outer layers depend inward.

```
            ┌──────────────────────────────────────┐
            │         Infrastructure               │
            │   (DB, APIs, File System, Messaging)  │
            │    ┌──────────────────────────────┐   │
            │    │       Application Layer      │   │
            │    │   (Use Cases, DTOs, Mappers) │   │
            │    │    ┌──────────────────┐      │   │
            │    │    │    Domain Layer  │      │   │
            │    │    │ (Entities, Value │      │   │
            │    │    │  Objects, Interfaces)│  │   │
            │    │    └──────────────────┘      │   │
            │    └──────────────────────────────┘   │
            └──────────────────────────────────────┘
           Dependencies flow INWARD
```

C# project structure for Clean Architecture:

```
src/
  MyApp.Domain/           # Entities, Value Objects, Domain Interfaces
    Entities/
    ValueObjects/
```

```
    Interfaces/             # IOrderRepository (interface only, no implementation)
  MyApp.Application/      # Use Cases, DTOs, Validators
    Orders/
      CreateOrderCommand.cs
      CreateOrderHandler.cs
      OrderDto.cs
  MyApp.Infrastructure/   # Implementations of domain interfaces
    Persistence/
      OrderRepository.cs    # Implements IOrderRepository
    ExternalServices/
  MyApp.WebApi/           # Controllers, Middleware, DI Setup
    Controllers/
    Program.cs
```

Key difference: In layered architecture, business logic depends on the data access layer. In Clean Architecture, the domain is independent — data access depends on domain interfaces (Dependency Inversion).

When to choose:

- **Layered:** Small CRUD apps, prototypes, teams new to architecture patterns. Simple and well-understood.
- **Clean Architecture:** Complex domain logic, need to swap infrastructure (e.g., different databases for different clients), long-lived products, large teams.

What to listen for: Understands that the key differentiator is the direction of dependencies, not the number of layers. Mentions that Clean Architecture's main benefit is domain isolation. Does not treat them as "bad vs good" — acknowledges layered architecture is appropriate for simpler systems.

C2. Explain Entities, Value Objects, and Aggregates in Domain-Driven Design.

Expected Answer:

Entity — Has a unique identity that persists over time. Two entities with the same data but different IDs are different.

```
public class TaxReturn
{
    public int Id { get; private set; }               // Identity
    public string TaxpayerId { get; private set; }
    public int TaxYear { get; private set; }
    public FilingStatus Status { get; private set; }
    public Money TotalIncome { get; private set; }

    public void Submit()
    {
        if (Status != FilingStatus.Draft)
            throw new DomainException("Only draft returns can be submitted.");
        Status = FilingStatus.Submitted;
    }
}
```

Value Object — Defined by its attributes, not identity. Two Value Objects with the same data are equal. Immutable.

```
public class Money : IEquatable<Money>
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency)
    {
        Amount = amount;
        Currency = currency;
    }

    public Money Add(Money other)
    {
        if (Currency != other.Currency)
            throw new DomainException("Cannot add different currencies.");
        return new Money(Amount + other.Amount, Currency);
    }

    public bool Equals(Money other)
        => other != null && Amount == other.Amount && Currency == other.Currency;

    public override int GetHashCode() => HashCode.Combine(Amount, Currency);
}
```

Aggregate — A cluster of entities and value objects treated as a unit for data changes. Has a root entity (Aggregate Root) that controls all access.

```
// TaxReturn is the Aggregate Root
public class TaxReturn  // Aggregate Root
{
    public int Id { get; private set; }
    private readonly List<W2Form> _w2Forms = new();    // Child entity
    private readonly List<Deduction> _deductions = new(); // Child entity
    public IReadOnlyList<W2Form> W2Forms => _w2Forms.AsReadOnly();
    public IReadOnlyList<Deduction> Deductions => _deductions.AsReadOnly();

    public void AddW2(W2Form w2)
    {
        if (_w2Forms.Any(f => f.EmployerEin == w2.EmployerEin))
            throw new DomainException("Duplicate W-2 for this employer.");
        _w2Forms.Add(w2);
        RecalculateTotalIncome();
    }

    public void RemoveW2(int w2Id)
```

```
    {
        var w2 = _w2Forms.SingleOrDefault(f => f.Id == w2Id)
            ?? throw new DomainException("W-2 not found.");
        _w2Forms.Remove(w2);
        RecalculateTotalIncome();
    }

    private void RecalculateTotalIncome()
    {
        TotalIncome = new Money(
            _w2Forms.Sum(f => f.Wages.Amount), "USD");
    }
}
```

Rules:

- External code accesses child entities only through the Aggregate Root.
- One transaction = one Aggregate. Cross-aggregate changes require eventual consistency.
- Keep aggregates small — include only entities that must change together.

What to listen for: Clear distinction between identity-based (Entity) and value-based (Value Object) equality. Understands the Aggregate Root as a consistency boundary. Mentions immutability for Value Objects.

Gate 3 — Scenario-Based

C3. You are designing a REST API that serves both a web app and mobile clients. The mobile team needs lighter responses, and you need to version the API without breaking existing clients. How do you approach this?

Expected Answer:

API Versioning strategies:

Strategy	Pros	Cons
URL path (`/api/v2/orders`)	Explicit, easy to route	URL pollution, hard to sunset
Query string (`?api-version=2`)	Non-breaking, optional	Easy to miss, caching issues
Header (`X-Api-Version: 2`)	Clean URLs	Hidden, harder to test in browser
Media type (`Accept: application/vnd.myapp.v2+json`)	RESTful, content negotiation	Complex, unfamiliar to many devs

Recommended approach for this scenario: URL path versioning for major versions, with response shaping per client.

```
// Use ASP.NET API Versioning
[ApiController]
[Route("api/v{version:apiVersion}/[controller]")]
[ApiVersion("1.0")]
[ApiVersion("2.0")]
public class OrdersController : ControllerBase
{
    [HttpGet("{id}")]
    [MapToApiVersion("1.0")]
    public async Task<ActionResult<OrderResponseV1>> GetV1(int id)
    {
        var order = await _orderService.GetByIdAsync(id);
        return Ok(MapToV1(order));
    }

    [HttpGet("{id}")]
    [MapToApiVersion("2.0")]
    public async Task<ActionResult<OrderResponseV2>> GetV2(int id)
    {
        var order = await _orderService.GetByIdAsync(id);
        return Ok(MapToV2(order));
    }
}
```

For lighter mobile responses, use response shaping:

```
// Option 1: Separate DTOs
public class OrderSummaryDto  // Mobile
{
    public int Id { get; set; }
    public string Status { get; set; }
    public decimal Total { get; set; }
}

public class OrderDetailDto  // Web
{
    public int Id { get; set; }
    public string Status { get; set; }
    public decimal Total { get; set; }
    public List<OrderLineItemDto> LineItems { get; set; }
    public AuditInfoDto AuditInfo { get; set; }
}

// Option 2: Field selection via query parameter
[HttpGet("{id}")]
public async Task<ActionResult> Get(int id, [FromQuery] string fields)
{
    var order = await _orderService.GetByIdAsync(id);
    if (!string.IsNullOrEmpty(fields))
        return Ok(ShapeResponse(order, fields.Split(',')));
```

```
    return Ok(order);
}
```

What to listen for: Does not conflate versioning with response shaping — these are separate concerns. Discusses deprecation strategy (how to sunset old versions). Mentions API documentation (Swagger/OpenAPI) as part of the versioning story. Considers backward compatibility and contract testing.

Red flag: Only knows URL versioning. Does not discuss how to handle breaking changes. Suggests "just add v2 when things change" without a versioning policy.

C4. Explain how you would implement the Mediator pattern (e.g., MediatR) in a .NET project. What are its benefits and when does it become problematic?

Expected Answer:

MediatR implements the Mediator pattern — instead of controllers calling services directly, they send request objects to a mediator, which routes them to the correct handler.

```
// Command
public class CreateTaxReturnCommand : IRequest<int>
{
    public string TaxpayerId { get; set; }
    public int TaxYear { get; set; }
    public string FilingStatus { get; set; }
}

// Handler
public class CreateTaxReturnHandler : IRequestHandler<CreateTaxReturnCommand, int>
{
    private readonly ITaxReturnRepository _repo;
    private readonly IValidator<CreateTaxReturnCommand> _validator;

    public CreateTaxReturnHandler(
        ITaxReturnRepository repo,
        IValidator<CreateTaxReturnCommand> validator)
    {
        _repo = repo;
        _validator = validator;
    }

    public async Task<int> Handle(
        CreateTaxReturnCommand request, CancellationToken ct)
    {
        await _validator.ValidateAndThrowAsync(request, ct);

        var taxReturn = new TaxReturn(
            request.TaxpayerId, request.TaxYear, request.FilingStatus);

        return await _repo.AddAsync(taxReturn);
    }
```

```
}

// Controller — thin, just dispatches
[HttpPost]
public async Task<ActionResult<int>> Create(CreateTaxReturnCommand command)
{
    var id = await _mediator.Send(command);
    return CreatedAtAction(nameof(GetById), new { id }, id);
}

// Pipeline behavior — cross-cutting concerns
public class LoggingBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
{
    private readonly ILogger _logger;

    public LoggingBehavior(ILogger logger) => _logger = logger;

    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken ct)
    {
        _logger.LogInformation("Handling {RequestType}", typeof(TRequest).Name);
        var response = await next();
        _logger.LogInformation("Handled {RequestType}", typeof(TRequest).Name);
        return response;
    }
}
```

Benefits:

- Thin controllers with no business logic
- Pipeline behaviors for cross-cutting concerns (logging, validation, caching)
- Each handler is independently testable with clear input/output
- CQRS natural fit — separate command and query handlers

When it becomes problematic:

- **Indirection tax** — Navigating from controller to handler requires searching by type, not following method calls. "Go to definition" does not work.
- **Simple CRUD overhead** — For basic CRUD operations, handler + command + validator classes are more code than a direct service call.
- **God pipeline** — Too many behaviors in the pipeline make debugging difficult.
- **Distributed monolith trap** — MediatR is in-process only; teams sometimes confuse it with a message bus and build false assumptions about decoupling.

What to listen for: Balanced view — understands both the benefits and the costs. Mentions pipeline behaviors as the key feature beyond simple dispatch. Knows that MediatR is in-process (not a message bus). Can articulate when the indirection cost outweighs the benefits.

Red flag: "MediatR is always the right choice for any project." Cannot explain what problem it solves beyond "clean controllers." Does not know about pipeline behaviors.

C5. You are tasked with breaking a monolithic .NET API into microservices. What factors drive your decomposition decisions, and what pitfalls do you anticipate?

Expected Answer:

Decomposition drivers:

1. **Domain boundaries** — Use bounded contexts from DDD. Each microservice owns one bounded context (e.g., Tax Preparation, E-Filing, User Management, Billing).
2. **Data ownership** — Each service owns its data store. If two features share the same tables heavily, they probably belong in the same service.
3. **Team structure** — Conway's Law: service boundaries should align with team boundaries. A service owned by two teams is a service owned by no team.
4. **Deployment cadence** — Components that change at different rates benefit from separation (e.g., tax calculation rules change annually; user management rarely changes).
5. **Scaling requirements** — Components with different resource profiles (CPU-bound calculation vs I/O-bound document storage) benefit from independent scaling.

Pitfalls:

Pitfall	Description	Mitigation
Distributed monolith	Services that must deploy together defeat the purpose	Ensure each service can deploy independently
Data consistency	No cross-service transactions	Saga pattern, eventual consistency, outbox pattern
Chatty communication	Too many synchronous inter-service calls	Aggregate data at service boundaries; use async messaging
Shared database	Multiple services reading the same tables	Each service gets its own schema/database
Over-decomposition	Too many tiny services for a small team	Start coarse, split when pain is evident
Observability	Request flows across multiple services	Distributed tracing (OpenTelemetry), correlation IDs

Migration strategy:

- **Strangler Fig** — Extract one bounded context at a time from the monolith.
- Start with the least coupled component.
- Keep the monolith running; route traffic to the new service via API gateway.

- Extract database tables last (hardest part).

What to listen for: Starts with domain boundaries, not technology. Mentions data ownership as a key challenge. Discusses eventual consistency rather than trying to maintain distributed transactions. Proposes incremental migration. Understands that microservices add operational complexity and are not always the right answer.

Red flag: Proposes decomposing by technical layer (one service for all controllers, one for all business logic). Plans a big-bang migration. Does not mention data ownership challenges. "Just use Kafka for everything."

↑ Back to Index

PART D — Testing

Gate 1 — Conceptual

D1. What are the different types of test doubles (mock, stub, fake, spy)? When do you use each?

Expected Answer:

Type	Purpose	Example
Stub	Returns predetermined data; does not verify calls	A `StubTaxRateProvider` that always returns 25%
Mock	Verifies that specific interactions occurred	Verify that `_emailSender.Send()` was called with correct arguments
Fake	Working implementation but simplified (not production)	In-memory database instead of SQL Server
Spy	Records calls for later inspection without replacing behavior	Wraps real logger to capture log entries for assertions

```
// Stub — provides canned data
var stubRepo = new Mock<ITaxReturnRepository>();
stubRepo.Setup(r => r.GetByIdAsync(42))
    .ReturnsAsync(new TaxReturn { Id = 42, Status = "Draft" });

// Mock — verifies interaction
var mockNotifier = new Mock<INotificationService>();
// ... execute code ...
mockNotifier.Verify(
    n => n.SendAsync("user@email.com", It.Is<string>(s =>
s.Contains("submitted"))),
    Times.Once);

// Fake — working but simplified implementation
public class InMemoryTaxReturnRepository : ITaxReturnRepository
```

```
{
    private readonly Dictionary<int, TaxReturn> _store = new();
    private int _nextId = 1;

    public Task<TaxReturn> GetByIdAsync(int id)
        => Task.FromResult(_store.GetValueOrDefault(id));

    public Task<int> AddAsync(TaxReturn taxReturn)
    {
        taxReturn.Id = _nextId++;
        _store[taxReturn.Id] = taxReturn;
        return Task.FromResult(taxReturn.Id);
    }
}

// Spy — records calls
public class SpyLogger : ILogger
{
    public List<string> LoggedMessages { get; } = new();

    public void LogInformation(string message, params object[] args)
    {
        LoggedMessages.Add(string.Format(message, args));
    }
}
```

When to use each:

- **Stub:** When the test needs specific return values but does not care about how the dependency was called.
- **Mock:** When the behavior being tested is the interaction itself (e.g., "did we send the email?").
- **Fake:** Integration-style tests that need realistic behavior without external infrastructure.
- **Spy:** When you want the real behavior but also need to inspect what happened.

What to listen for: Clear distinction between "state verification" (stubs/fakes) and "behavior verification" (mocks). Understands that over-mocking leads to brittle tests. Mentions that most tests should prefer stubs; mocks are for verifying side effects.

D2. Explain the Test Pyramid. What should you test at each level?

Expected Answer:

```
          /\
         /  \
        / E2E\          Few: Critical user journeys
       /------\
      /  Integ \        Moderate: Component integration, API contracts
     /----------\
    /   Unit     \      Many: Business logic, edge cases, algorithms
   /--------------\
```

Level	Count	Speed	What to Test
Unit	Many (70-80%)	Fast (ms)	Business rules, calculations, validation logic, edge cases, error handling
Integration	Moderate (15-20%)	Medium (sec)	Database queries, API endpoints, middleware pipeline, DI container wiring
E2E	Few (5-10%)	Slow (min)	Critical user flows, smoke tests for deployment verification

What NOT to test (unit level):

- Private methods directly (test through public API)
- Framework code (ASP.NET routing, DI container)
- Trivial getters/setters
- Third-party library internals

Example at each level:

```
// Unit test — tests calculation logic in isolation
[Test]
public void CalculateTax_ShouldApplyStandardDeduction_WhenNoItemizedDeductions()
{
    //Arrange
    var calculator = new TaxCalculator();
    var income = 75_000m;

    //Act
    var result = calculator.CalculateTaxableIncome(income, deductions: null);

    //Assert
    result.Should().Be(75_000m - 13_850m); // Standard deduction applied
}

// Integration test — tests API endpoint with real middleware
[Fact]
public async Task GetTaxReturn_ShouldReturn200_WhenReturnExists()
{
    //Arrange
    var client = _factory.CreateClient();

    //Act
    var response = await client.GetAsync("/api/v3/taxreturns/42");

    //Assert
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    var body = await response.Content.ReadFromJsonAsync<TaxReturnDto>();
    body.Id.Should().Be(42);
}
```

What to listen for: Describes the pyramid shape as an economic decision (fast, cheap tests at the bottom; slow, expensive tests at the top). Understands that inverting the pyramid (many E2E, few unit) leads to slow, flaky CI. Mentions that the exact ratios depend on the application (CRUD-heavy apps may have more integration tests).

Gate 2 — Applied

D3. How do you test async code and controllers in .NET? Show examples with Moq and NUnit or xUnit.

Expected Answer:

Testing async service methods:

```
[TestFixture]
public class TaxReturnServiceTests
{
    private AutoMocker _mocker;
    private TaxReturnService _sut;

    [SetUp]
    public void SetUp()
    {
        _mocker = new AutoMocker();
        _sut = _mocker.CreateInstance<TaxReturnService>();
    }

    [Test]
    public async Task GetByIdAsync_ShouldReturnTaxReturn_WhenExists()
    {
        //Arrange
        var expected = new TaxReturn { Id = 42, TaxYear = 2025 };
        _mocker.GetMock<ITaxReturnRepository>()
            .Setup(r => r.GetByIdAsync(42))
            .ReturnsAsync(expected);

        //Act
        var result = await _sut.GetByIdAsync(42);

        //Assert
        result.Should().NotBeNull();
        result.Id.Should().Be(42);
        result.TaxYear.Should().Be(2025);
    }

    [Test]
    public async Task GetByIdAsync_ShouldThrow_WhenNotFound()
    {
        //Arrange
        _mocker.GetMock<ITaxReturnRepository>()
```

```
            .Setup(r => r.GetByIdAsync(It.IsAny<int>()))
            .ReturnsAsync((TaxReturn)null);

        //Act
        Func<Task> act = async () => await _sut.GetByIdAsync(999);

        //Assert
        await act.Should().ThrowAsync<NotFoundException>()
            .WithMessage("*999*");
    }

    [Test]
    public async Task CreateAsync_ShouldCallRepositoryAndNotify()
    {
        //Arrange
        var command = new CreateTaxReturnCommand
        {
            TaxpayerId = "123-45-6789",
            TaxYear = 2025
        };
        _mocker.GetMock<ITaxReturnRepository>()
            .Setup(r => r.AddAsync(It.IsAny<TaxReturn>()))
            .ReturnsAsync(42);

        //Act
        var result = await _sut.CreateAsync(command);

        //Assert
        result.Should().Be(42);
        _mocker.GetMock<ITaxReturnRepository>()
            .Verify(r => r.AddAsync(It.Is<TaxReturn>(
                t => t.TaxpayerId == "123-45-6789" && t.TaxYear == 2025)),
                Times.Once);
        _mocker.GetMock<INotificationService>()
            .Verify(n => n.SendAsync(It.IsAny<string>(), It.IsAny<string>()),
                Times.Once);
    }
}
```

Testing controllers (xUnit with WebApplicationFactory):

```
public class TaxReturnControllerTests :
IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public TaxReturnControllerTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.WithWebHostBuilder(builder =>
        {
            builder.ConfigureServices(services =>
            {
```

```
                // Replace real service with mock
                services.AddScoped<ITaxReturnService>(_ =>
                {
                    var mock = new Mock<ITaxReturnService>();
                    mock.Setup(s => s.GetByIdAsync(42))
                        .ReturnsAsync(new TaxReturnDto { Id = 42 });
                    return mock.Object;
                });
            });
        }).CreateClient();
    }

    [Fact]
    public async Task Get_ShouldReturn200WithTaxReturn()
    {
        //Act
        var response = await _client.GetAsync("/api/v3/taxreturns/42");

        //Assert
        response.StatusCode.Should().Be(HttpStatusCode.OK);
    }

    [Fact]
    public async Task Get_ShouldReturn404_WhenNotFound()
    {
        //Act
        var response = await _client.GetAsync("/api/v3/taxreturns/999");

        //Assert
        response.StatusCode.Should().Be(HttpStatusCode.NotFound);
    }
}
```

Key async testing pitfalls:

- `async void` test methods silently swallow exceptions — always return `Task`.
- Use `ReturnsAsync()` for setting up async mock methods.
- Use `await act.Should().ThrowAsync<T>()` for testing exceptions in async code.

What to listen for: Uses `AutoMocker` for automatic dependency resolution. Tests both happy path and error cases. Async tests return `Task`, not `void`. For controller tests, mentions the choice between unit testing (mock service directly) vs integration testing (WebApplicationFactory).

Red flag: Uses `async void` tests. Uses `.Result` or `.Wait()` in tests. Only tests the happy path. Does not know about `AutoMocker`.

Gate 3 — Scenario-Based

D4. A test suite has 500 tests that take 15 minutes to run. Many tests are flaky. Your team avoids writing new tests because the suite is unreliable. How do you rehabilitate it?

Expected Answer:

Phase 1: Triage (Week 1)

1. Run the suite 5 times, identify all flaky tests (different results across runs).
2. Tag flaky tests with `[Category("Flaky")]` and exclude from CI gate (but still run them separately).
3. Analyze flaky tests — most fall into these categories:
    - Time-dependent (`DateTime.Now`)
    - Order-dependent (shared state between tests)
    - External dependency (network, database, file system)
    - Race conditions (async tests not awaited properly)

Phase 2: Fix structural issues (Weeks 2-3)

1. **Shared state** — Ensure each test creates its own data. Use `[SetUp]`/`[TearDown]` to reset state.
2. **Time dependency** — Inject `IClock` or `TimeProvider` instead of `DateTime.Now`.
3. **External dependencies** — Replace with fakes for unit tests; use containers (Testcontainers) for integration tests.
4. **Parallelization** — Fix tests that fail when run in parallel; then enable parallel execution.

Phase 3: Speed (Weeks 3-4)

1. Identify slow tests (add timing to CI output).
2. Move integration tests to a separate project that runs less frequently.
3. Use in-memory database for tests that do not specifically test SQL behavior.
4. Delete tests that test framework behavior (e.g., "does DI container resolve?").

Phase 4: Culture (ongoing)

1. CI must block on test failure — no "just re-run it" culture.
2. New code requires tests — enforce in code review.
3. Flaky test budget: if a test fails nondeterministically more than twice, fix or delete it.

What to listen for: Starts with triage and quarantine, not "rewrite everything." Identifies common flakiness causes. Addresses both technical (parallelization, fakes) and cultural (CI enforcement) aspects. Mentions concrete timeline expectations.

Red flag: "Just delete all the flaky tests." No mention of root cause analysis. Proposes rewriting 500 tests from scratch. Does not address the cultural problem.

D5. Describe TDD (Red-Green-Refactor). Walk through developing a tax bracket calculator using TDD.

Expected Answer:

TDD cycle:

1. **Red** — Write a failing test for the simplest case.
2. **Green** — Write the minimum code to make it pass.
3. **Refactor** — Clean up without changing behavior.

Tax bracket calculator — step by step:

```
// RED: Simplest case — zero income
[Test]
public void CalculateTax_ShouldReturnZero_WhenIncomeIsZero()
{
    //Arrange
    var calculator = new TaxBracketCalculator();

    //Act
    var tax = calculator.Calculate(0m);

    //Assert
    tax.Should().Be(0m);
}

// GREEN: Minimum implementation
public class TaxBracketCalculator
{
    public decimal Calculate(decimal income) => 0m;
}
```

```
// RED: First bracket — 10% on income up to $11,000
[Test]
public void CalculateTax_ShouldApply10Percent_WhenIncomeWithinFirstBracket()
{
    //Arrange
    var calculator = new TaxBracketCalculator();

    //Act
    var tax = calculator.Calculate(10_000m);

    //Assert
    tax.Should().Be(1_000m); // 10,000 * 0.10
}

// GREEN: Handle first bracket
public decimal Calculate(decimal income)
{
    if (income <= 0) return 0m;
    if (income <= 11_000m) return income * 0.10m;
    return 0m; // Will fail next test — that's intentional
}
```

```
// RED: Second bracket — 12% on $11,001 to $44,725
[Test]
public void CalculateTax_ShouldApplyBrackets_WhenIncomeSpansTwoBrackets()
{
```

```
        //Arrange
        var calculator = new TaxBracketCalculator();

        //Act
        var tax = calculator.Calculate(30_000m);

        //Assert
        // First $11,000 at 10% = $1,100
        // Remaining $19,000 at 12% = $2,280
        // Total = $3,380
        tax.Should().Be(3_380m);
    }

    // GREEN: Build up bracket logic
    public decimal Calculate(decimal income)
    {
        if (income <= 0) return 0m;

        decimal tax = 0m;
        decimal remaining = income;

        // 10% bracket: $0 - $11,000
        var bracketAmount = Math.Min(remaining, 11_000m);
        tax += bracketAmount * 0.10m;
        remaining -= bracketAmount;

        // 12% bracket: $11,001 - $44,725
        if (remaining > 0)
        {
            bracketAmount = Math.Min(remaining, 33_725m);
            tax += bracketAmount * 0.12m;
            remaining -= bracketAmount;
        }

        return tax;
    }

    // REFACTOR: Extract brackets into a data structure
    public class TaxBracketCalculator
    {
        private static readonly (decimal Limit, decimal Rate)[] Brackets =
        {
            (11_000m, 0.10m),
            (44_725m, 0.12m),
            (95_375m, 0.22m),
            (182_100m, 0.24m),
            (231_250m, 0.32m),
            (578_125m, 0.35m),
            (decimal.MaxValue, 0.37m)
        };

        public decimal Calculate(decimal income)
        {
            if (income <= 0) return 0m;
```

```
        decimal tax = 0m;
        decimal previousLimit = 0m;

        foreach (var (limit, rate) in Brackets)
        {
            if (income <= previousLimit) break;

            var taxableInBracket = Math.Min(income, limit) - previousLimit;
            tax += taxableInBracket * rate;
            previousLimit = limit;
        }

        return tax;
    }
}
```

What to listen for: Follows the cycle strictly — does not write production code before a failing test. Each test drives one small increment. Refactoring step generalizes without adding new behavior. Mentions that TDD works best for algorithmic/logic-heavy code and may be less natural for I/O-heavy code.

Red flag: Writes all tests first, then all code. Skips the refactoring step. Writes tests that test implementation details rather than behavior. Says "TDD is too slow, I test after."

↑ Back to Index

PART E — Cloud & DevOps

Gate 2 — Applied

E1. Compare Azure App Service, Azure Functions, and containerized deployments (AKS/ACA). When would you choose each for a .NET API?

Expected Answer:

Factor	App Service	Azure Functions	Container (AKS/ACA)
Best for	Standard web APIs, always-on services	Event-driven, sporadic workloads	Complex systems, multi-service
Scaling	Manual or auto-scale rules	Consumption plan: scale to zero	Horizontal pod autoscaling
Cost	Predictable monthly	Pay-per-execution	Pay for cluster capacity
Cold start	None (always running)	Yes on Consumption plan	Depends on config
Complexity	Low	Low-Medium	High

Factor	App Service	Azure Functions	Container (AKS/ACA)
Max execution	Unlimited	10 min (Consumption), 60 min (Premium)	Unlimited

Decision framework:

- **App Service:** Default choice for a .NET API. Always-on, managed TLS, deployment slots for blue-green, integrated with Azure AD. Choose when you have a standard web API with predictable traffic.
- **Azure Functions:** Event processing (Service Bus triggers, blob triggers), scheduled tasks, lightweight APIs with sporadic traffic. Choose when workload is event-driven or you want scale-to-zero.
- **Container (AKS):** Multiple services with different technology stacks, need fine-grained networking control, compliance requires specific OS configurations. Choose when you have 5+ microservices that need orchestration.
- **Azure Container Apps (ACA):** Serverless containers — the middle ground. Choose when you want container flexibility without Kubernetes operational complexity.

What to listen for: Decision is driven by workload characteristics, not technology preference. Mentions cost implications. Understands that AKS introduces significant operational overhead. Knows about App Service deployment slots for zero-downtime deployments.

E2. Explain the key differences between Azure Service Bus and AWS SQS/SNS. When would you use a message queue vs a pub/sub topic?

Expected Answer:

Feature	Azure Service Bus	AWS SQS	AWS SNS
Type	Queue + Topic	Queue only	Pub/Sub only
Message ordering	FIFO (sessions)	FIFO (optional)	No ordering guarantee
Message size	256 KB (Standard), 100 MB (Premium)	256 KB	256 KB
Dead-letter queue	Built-in	Built-in	Via SQS subscription
Transactions	Yes (send + complete atomically)	No	No
Duplicate detection	Built-in	Dedup on FIFO queues	No

Queue vs Pub/Sub:

```
Queue (Point-to-Point):
  Producer --> [Queue] --> Consumer
  Each message consumed by ONE consumer.
  Use for: task distribution, work queues, command processing.
```

```
Pub/Sub (Topic):
  Publisher --> [Topic] --> Subscriber A
                        --> Subscriber B
                        --> Subscriber C
  Each message delivered to ALL subscribers.
  Use for: event notification, fan-out, decoupled systems.
```

When to use which:

- **Queue:** Processing a tax return submission (one processor handles each return). Order processing pipeline. Background job execution.
- **Pub/Sub:** Tax return status changed (notify UI, audit log, analytics, and email service independently). User registered event (create profile, send welcome email, provision resources).

Combined pattern (common): SNS topic fans out to multiple SQS queues, each consumed by a different service.

```
 Tax Return Submitted (SNS)
       |          |           |
  [SQS: PDF]  [SQS: Email]  [SQS: Analytics]
       |          |           |
  PDF Svc     Email Svc    Analytics Svc
```

What to listen for: Understands the fundamental difference between point-to-point and pub/sub. Gives concrete use cases for each. Mentions dead-letter queues for error handling. Knows the SNS+SQS fan-out pattern.

Gate 3 — Scenario-Based

E3. Write a Dockerfile for a .NET 8 API with multi-stage build. Explain each stage and why multi-stage matters.

Expected Answer:

```dockerfile
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy csproj files first for layer caching
COPY ["MyApi/MyApi.csproj", "MyApi/"]
COPY ["MyApi.Domain/MyApi.Domain.csproj", "MyApi.Domain/"]
COPY ["MyApi.Infrastructure/MyApi.Infrastructure.csproj", "MyApi.Infrastructure/"]
RUN dotnet restore "MyApi/MyApi.csproj"

# Copy everything else and build
COPY . .
```

```dockerfile
RUN dotnet publish "MyApi/MyApi.csproj" \
    -c Release \
    -o /app/publish \
    --no-restore

# Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# Security: run as non-root user
RUN adduser --disabled-password --gecos "" appuser
USER appuser

COPY --from=build /app/publish .

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s \
    CMD curl -f http://localhost:8080/health || exit 1

EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080

ENTRYPOINT ["dotnet", "MyApi.dll"]
```

Why multi-stage:

1. **Image size** — SDK image is ~700MB; runtime image is ~220MB. The final image only contains the runtime and published output.
2. **Security** — No build tools (compilers, package managers) in production image means smaller attack surface.
3. **Build caching** — Copying `.csproj` files first means the `dotnet restore` layer is cached unless dependencies change.
4. **Reproducibility** — The build environment is defined in the Dockerfile, not dependent on the developer's machine.

docker-compose for local development:

```yaml
version: '3.8'
services:
  api:
    build:
      context: .
      dockerfile: MyApi/Dockerfile
    ports:
      - "5000:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ConnectionStrings__DefaultConnection=Server=db;Database=MyApp;...
    depends_on:
      db:
        condition: service_healthy
```

```
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=DevPassword123!
    ports:
      - "1433:1433"
    healthcheck:
      test: /opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P DevPassword123! -Q
"SELECT 1"
      interval: 10s
      timeout: 3s
      retries: 5
```

What to listen for: Understands layer caching optimization (copy csproj first). Knows the security implications (non-root user, minimal runtime image). Can explain the size difference between SDK and runtime images. Mentions health checks.

Red flag: Single-stage build with SDK image. No layer caching strategy. Runs as root. Does not know what multi-stage means.

E4. Describe blue-green and canary deployment strategies. How would you implement canary for a .NET API on Kubernetes?

Expected Answer:

Blue-Green: Two identical environments. Green is live; Blue has the new version. Switch traffic at once via load balancer. Instant rollback by switching back.

```
          ┌──────────┐
Traffic   │  Load    │
-------->| Balancer |
          └──────┬───┘
                 |
      ┌──────────┼──────────┐
      v                     v
[Green: v1.0]      [Blue: v1.1]
 (LIVE)             (Staged)

After validation, switch:
[Green: v1.0]      [Blue: v1.1]
 (Idle)             (LIVE)
```

Canary: Gradually shift traffic percentage from old to new version. Monitor error rates and latency at each step. Roll back if metrics degrade.

```
Step 1:  95% --> v1.0    5% --> v1.1  (canary)
Step 2:  80% --> v1.0   20% --> v1.1
Step 3:  50% --> v1.0   50% --> v1.1
Step 4:   0% --> v1.0  100% --> v1.1  (full rollout)
```

Kubernetes canary implementation:

```yaml
# Stable deployment (v1.0)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tax-api-stable
spec:
  replicas: 9
  selector:
    matchLabels:
      app: tax-api
      track: stable
  template:
    metadata:
      labels:
        app: tax-api
        track: stable
    spec:
      containers:
      - name: tax-api
        image: myregistry/tax-api:1.0.0
        ports:
        - containerPort: 8080

---
# Canary deployment (v1.1)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tax-api-canary
spec:
  replicas: 1  # 1 out of 10 total = 10% traffic
  selector:
    matchLabels:
      app: tax-api
      track: canary
  template:
    metadata:
      labels:
        app: tax-api
        track: canary
    spec:
      containers:
      - name: tax-api
        image: myregistry/tax-api:1.1.0
```

```yaml
        ports:
        - containerPort: 8080

---
# Service selects both stable and canary pods
apiVersion: v1
kind: Service
metadata:
  name: tax-api
spec:
  selector:
    app: tax-api  # Matches BOTH stable and canary
  ports:
  - port: 80
    targetPort: 8080
```

For more precise traffic splitting, use a service mesh (Istio) or ingress controller:

```yaml
# Istio VirtualService for precise traffic splitting
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: tax-api
spec:
  hosts:
  - tax-api
  http:
  - route:
    - destination:
        host: tax-api
        subset: stable
      weight: 90
    - destination:
        host: tax-api
        subset: canary
      weight: 10
```

What to listen for: Understands the trade-off: blue-green is simpler but requires 2x resources; canary is more gradual but more complex. Mentions monitoring as integral to canary (not just deployment). Knows that Kubernetes native approach gives approximate traffic splitting (replica ratio) while Istio gives precise control.

Red flag: Confuses blue-green with canary. Cannot explain rollback procedure. Does not mention monitoring during canary rollout.

E5. What are the 12-factor app principles? Which ones are most relevant to a .NET cloud deployment?

Expected Answer:

Factor	Principle	.NET Relevance
I. Codebase	One codebase tracked in version control	Git repo; multiple deployments via configuration
II. Dependencies	Explicitly declare and isolate	NuGet packages in `.csproj`; no GAC dependencies
III. Config	Store config in environment	`IConfiguration` with env vars, Azure Key Vault
IV. Backing services	Treat as attached resources	Connection strings via config, not hard-coded
V. Build, release, run	Strictly separate stages	CI builds artifact; CD releases to environment
VI. Processes	Execute as stateless processes	No in-process session state; use Redis/SQL
VII. Port binding	Export services via port binding	Kestrel binds to port; no IIS dependency
VIII. Concurrency	Scale out via process model	Multiple pods/instances, not bigger VMs
IX. Disposability	Fast startup, graceful shutdown	`IHostedService`, `CancellationToken`, health checks
X. Dev/prod parity	Keep environments similar	Docker ensures parity; feature flags not env branches
XI. Logs	Treat as event streams	Serilog to stdout; aggregator (Datadog/ELK) collects
XII. Admin processes	Run admin tasks as one-off processes	EF migrations as CI step, not at startup

Most relevant for .NET cloud deployments:

1. **III. Config** — `appsettings.json` for defaults, environment variables for overrides, Key Vault for secrets. Never hard-code connection strings.

```
builder.Configuration
    .AddJsonFile("appsettings.json")
    .AddJsonFile($"appsettings.{env}.json", optional: true)
    .AddEnvironmentVariables()
    .AddAzureKeyVault(new Uri(vaultUrl), new DefaultAzureCredential());
```

2. **VI. Processes (Statelessness)** — No sticky sessions, no in-memory cache as source of truth. Any instance can handle any request.

3. **IX. Disposability** — Graceful shutdown to finish in-flight requests.

```
app.Lifetime.ApplicationStopping.Register(() =>
{
    // Drain connections, flush logs
    Log.CloseAndFlush();
});
```

4. **XI. Logs** — Write to stdout/stderr, let infrastructure handle routing.

What to listen for: Can name at least 6-7 factors from memory. Connects each to concrete .NET practices. Understands that these principles enable horizontal scaling and cloud portability. Mentions that some factors (like codebase, dependencies) are already natural in modern .NET projects.

Red flag: Cannot name more than 2-3 factors. Says "we don't need 12-factor because we use Azure." Confuses the factors with microservice patterns.

↑ Back to Index

PART F — Git & Version Control

Gate 1 — Conceptual

F1. Explain the difference between `git merge` and `git rebase`. When would you use each?

Expected Answer:

Merge — Creates a merge commit that combines two branches. Preserves the complete history of both branches.

```
Before merge:
     A---B---C  (feature)
    /
D---E---F---G  (main)

After git merge feature:
     A---B---C
    /         \
D---E---F---G---M  (main, merge commit)
```

Rebase — Replays commits from one branch onto another. Creates a linear history by rewriting commits.

```
Before rebase:
     A---B---C  (feature)
    /
D---E---F---G  (main)

After git rebase main (while on feature):
```

```
              A'--B'--C'  (feature, rewritten commits)
             /
D---E---F---G  (main)
```

When to use which:

Situation	Use	Why
Updating feature branch with latest main	Rebase	Clean linear history for the feature
Merging feature into main via PR	Merge (squash)	Preserves the PR as a single commit on main
Shared branch with multiple contributors	Merge	Rebase rewrites history, causing conflicts for others
Long-lived release branches	Merge	Need to preserve exact history for auditing
Local cleanup before pushing	Rebase (interactive)	Clean up WIP commits before sharing

Golden rule: Never rebase commits that have been pushed and shared with others. Rebase rewrites commit hashes, which forces others to reconcile diverged history.

What to listen for: Understands that rebase rewrites history (new commit hashes). Knows the golden rule about not rebasing shared commits. Can explain when each produces a cleaner workflow. Mentions squash merge as a common PR strategy.

F2. Describe GitFlow, trunk-based development, and GitHub Flow. Which do you prefer and why?

Expected Answer:

GitFlow:

```
main ──────────●──────────────────●──────── (releases only)
               \                 /
develop ───●───●───●───●───●──────────── (integration)
           |       |
feature/x ─●──●    |
                   feature/y ─●──●
```

- Long-lived `develop` branch; `main` for releases only.
- Release branches for stabilization.
- Hotfix branches off `main`.
- Pros: Structured release management. Cons: Complex, slow for CI/CD.

Trunk-Based Development:

```
main ──●──●──●──●──●──●──●──●──● (everyone commits here)
       |     |        |
       short-lived feature branches (hours/1-2 days max)
```

- Everyone works on `main` (trunk) with very short-lived branches.
- Feature flags hide incomplete work.
- Pros: Fast CI/CD, simple. Cons: Requires mature testing and feature flag infrastructure.

GitHub Flow:

```
main ──●──────●──────────●────── (always deployable)
       |      |          |
       └──PR──┘    └──PR──┘
       feature     feature
```

- `main` is always deployable.
- Feature branches off `main`, merged via PR.
- No `develop` or release branches.
- Pros: Simple, works well with CI/CD. Cons: No structured release process.

Preference depends on context:

- **Startup / SaaS with continuous deployment:** Trunk-based or GitHub Flow.
- **Enterprise with scheduled releases:** GitFlow (or modified GitFlow).
- **Open source:** GitHub Flow (PRs are the collaboration unit).

What to listen for: Can describe at least two strategies with their trade-offs. Choice is context-dependent, not dogmatic. Understands that GitFlow's complexity is justified only when you need structured release management. Mentions feature flags as an enabler for trunk-based development.

Gate 2 — Applied

F3. You are reviewing a PR and notice the branch has 47 commits including "WIP", "fix typo", "oops", and "actually fix it this time." How do you handle this as a reviewer? What Git techniques can clean this up?

Expected Answer:

As a reviewer:

1. Focus the review on the final diff (`Files changed` tab), not individual commits. The commit messages are noise.
2. Ask the author to squash before merge, or use the "Squash and merge" merge strategy.
3. For future PRs, suggest interactive rebase before pushing.

Git techniques to clean up:

```
# Interactive rebase to squash/reorder commits
git rebase -i main

# In the editor:
pick a1b2c3 Add user registration endpoint
squash d4e5f6 WIP
squash g7h8i9 fix typo
squash j0k1l2 oops
squash m3n4o5 actually fix it this time
pick p6q7r8 Add validation for registration
squash s9t0u1 fix validation edge case

# Result: 2 clean commits instead of 47
```

```
# Alternative: Squash merge (preserves original branch)
git checkout main
git merge --squash feature/user-registration
git commit -m "feat: add user registration endpoint with validation"
```

```
# Autosquash with fixup commits (planned cleanup)
# During development:
git commit -m "Add registration endpoint"
git commit --fixup HEAD   # Creates "fixup! Add registration endpoint"
git commit -m "Add validation"
git commit --fixup HEAD~1 # Creates "fixup! Add registration endpoint"

# Before PR:
git rebase -i --autosquash main
# fixup commits automatically squash into their targets
```

What to listen for: Does not refuse to review based on messy commits. Knows multiple approaches (interactive rebase, squash merge, fixup commits). Understands that squash merge loses individual commit granularity, which is sometimes a feature and sometimes a loss. Mentions team convention (some teams require clean commits before review; others squash at merge time).

Red flag: Insists on rejecting the PR solely for messy commits. Does not know about interactive rebase. Says "commit messages do not matter."

F4. Two developers working on the same file create conflicting changes. Walk through how you resolve a merge conflict, and how to minimize conflicts in the first place.

Expected Answer:

Resolving a merge conflict:

```
# Attempt merge
git merge feature/other-devs-branch

# Git output:
# CONFLICT (content): Merge conflict in TaxReturnService.cs
# Automatic merge failed; fix conflicts and then commit the result.

# View the conflict markers
git diff
```

```
// TaxReturnService.cs with conflict markers:
public decimal CalculateRefund(TaxReturn taxReturn)
{
<<<<<<< HEAD (your changes)
    var totalTax = _bracketCalculator.Calculate(taxReturn.TaxableIncome);
    var credits = _creditService.CalculateCredits(taxReturn);
    return taxReturn.WithheldAmount - totalTax + credits;
=======
    var totalTax = CalculateTotalTax(taxReturn);
    return taxReturn.TotalWithheld - totalTax;
>>>>>>> feature/other-devs-branch (their changes)
}
```

Resolution steps:

1. Understand both changes — your version adds credits; theirs renamed properties.
2. Combine both intents:

```
public decimal CalculateRefund(TaxReturn taxReturn)
{
    var totalTax = _bracketCalculator.Calculate(taxReturn.TaxableIncome);
    var credits = _creditService.CalculateCredits(taxReturn);
    return taxReturn.TotalWithheld - totalTax + credits; // Combined: their rename
+ your credits
}
```

3. Mark resolved and complete merge:

```
git add TaxReturnService.cs
git merge --continue
```

Minimizing conflicts:

1. **Small, frequent PRs** — Merge daily rather than weekly.
2. **Clear ownership** — If possible, different developers work on different files/modules.

3. **Pull before push** — Rebase onto latest main before creating PR.
4. **Communicate** — If you know two people are modifying the same file, coordinate.
5. **Architectural separation** — Clean Architecture naturally reduces file overlap (changes to domain logic rarely conflict with infrastructure changes).

What to listen for: Understands conflict markers and can manually resolve. Emphasizes understanding both changes before resolving (not just picking one side). Mentions prevention strategies. Knows the difference between merge conflicts and semantic conflicts (code compiles but logic is wrong).

Red flag: Always picks one side ("just accept theirs"). Does not know what conflict markers mean. No prevention strategies.

↑ Back to Index

PART G — Data Structures & Algorithms (Essentials)

Gate 1 — Conceptual

G1. Explain Big-O notation. What are the time complexities of common operations on .NET collections?

Expected Answer:

Big-O describes the upper bound of an algorithm's growth rate as input size increases. It measures worst-case scalability, not absolute speed.

Big-O	Name	Example
O(1)	Constant	Dictionary lookup, array index access
O(log n)	Logarithmic	Binary search, SortedSet lookup
O(n)	Linear	List.Contains, iterating an array
O(n log n)	Linearithmic	Sorting (Array.Sort, LINQ OrderBy)
O(n^2)	Quadratic	Nested loops, bubble sort
O(2^n)	Exponential	Brute-force subset generation

.NET collections and their complexities:

Collection	Add	Remove	Lookup	Iterate	Notes
`List<T>`	O(1) amortized	O(n)	O(n) by value, O(1) by index	O(n)	Backed by array
`Dictionary<K,V>`	O(1) avg	O(1) avg	O(1) avg	O(n)	Hash table; O(n) worst case on collision

Collection	Add	Remove	Lookup	Iterate	Notes
`HashSet<T>`	O(1) avg	O(1) avg	O(1) avg	O(n)	Unique elements
`SortedDictionary<K,V>`	O(log n)	O(log n)	O(log n)	O(n)	Red-black tree
`SortedSet<T>`	O(log n)	O(log n)	O(log n)	O(n)	Red-black tree
`Queue<T>`	O(1)	O(1)	N/A	O(n)	FIFO
`Stack<T>`	O(1)	O(1)	N/A	O(n)	LIFO
`LinkedList<T>`	O(1) at ends	O(1) with node	O(n)	O(n)	Rarely used in practice
`ConcurrentDictionary<K,V>`	O(1) avg	O(1) avg	O(1) avg	O(n)	Thread-safe, striped locks

Practical implication:

```
// O(n^2) — checking for duplicates with nested loop
for (int i = 0; i < list.Count; i++)
    for (int j = i + 1; j < list.Count; j++)
        if (list[i] == list[j]) return true;

// O(n) — same task with HashSet
var seen = new HashSet<int>();
foreach (var item in list)
    if (!seen.Add(item)) return true;
```

What to listen for: Explains Big-O in terms of growth rate, not absolute time. Maps .NET collection types to their underlying data structures (Dictionary = hash table, SortedDictionary = red-black tree). Gives a practical example of choosing the right collection.

G2. When would you use a Dictionary vs a List vs a HashSet in C#? Give scenarios for each.

Expected Answer:

Dictionary<TKey, TValue> — When you need to look up values by a key.

```
// Mapping tax form types to their handlers
var handlers = new Dictionary<string, IFormHandler>
{
    ["W2"] = new W2Handler(),
    ["1099-INT"] = new Form1099IntHandler(),
    ["1099-DIV"] = new Form1099DivHandler()
};
```

```
// O(1) lookup
var handler = handlers["W2"];
```

List — When you need ordered, indexed access and may have duplicates.

```
// Maintaining an ordered list of tax return line items
var lineItems = new List<TaxLineItem>();
lineItems.Add(new TaxLineItem { Line = 1, Amount = 85_000m });
lineItems.Add(new TaxLineItem { Line = 2, Amount = 13_850m });

// Access by index: O(1)
var firstItem = lineItems[0];

// Iterate in order
foreach (var item in lineItems.OrderBy(i => i.Line)) { }
```

HashSet — When you need unique elements and fast membership testing.

```
// Track which states a multi-state filer has returns for
var filedStates = new HashSet<string> { "CA", "NY", "TX" };

// O(1) membership check
if (filedStates.Contains("CA")) { /* ... */ }

// Set operations
var allStates = new HashSet<string> { "CA", "NY", "TX", "FL", "WA" };
var unfiledStates = allStates.Except(filedStates); // FL, WA
```

Decision guide:

Need	Use
Key-value lookup	`Dictionary<K,V>`
Ordered, indexed access	`List<T>`
Unique elements + fast Contains	`HashSet<T>`
Sorted unique elements	`SortedSet<T>`
Thread-safe key-value	`ConcurrentDictionary<K,V>`
FIFO processing	`Queue<T>`
LIFO / undo stack	`Stack<T>`

What to listen for: Chooses based on access patterns, not habit. Knows that `List.Contains()` is O(n) while `HashSet.Contains()` is O(1). Mentions `ConcurrentDictionary` for multi-threaded scenarios without prompting.

Gate 2 — Applied

G3. You have a list of 1 million tax returns and need to find all returns for a specific taxpayer. The operation is performed frequently. How do you optimize this?

Expected Answer:

Naive approach — O(n) each time:

```
// Scans entire list every time
var returns = allReturns.Where(r => r.TaxpayerId == taxpayerId).ToList();
```

Optimized — build a lookup once, O(1) per query:

```
// Build index once: O(n)
var returnsByTaxpayer = allReturns
    .GroupBy(r => r.TaxpayerId)
    .ToDictionary(
        g => g.Key,
        g => g.ToList());

// Query: O(1)
if (returnsByTaxpayer.TryGetValue(taxpayerId, out var taxpayerReturns))
{
    // Process taxpayerReturns
}
```

Alternative — ILookup<TKey, TElement> for immutable multi-value lookup:

```
var lookup = allReturns.ToLookup(r => r.TaxpayerId);
var returns = lookup[taxpayerId]; // Returns empty sequence if not found
```

If data is in a database (the real-world answer):

```
// Don't load 1M records into memory
// Use a parameterized query with an index on TaxpayerId
var returns = await db.FetchAsync<TaxReturn>(
    "WHERE TaxpayerId = @0", taxpayerId);
```

If the data must be in memory and is sorted:

```
// Binary search for sorted data: O(log n)
var index = allReturns.BinarySearch(
```

```
        new TaxReturn { TaxpayerId = taxpayerId },
        new TaxpayerIdComparer());
```

What to listen for: Immediately identifies that repeated linear scans are the problem. Suggests building an index (Dictionary/Lookup). Mentions the database-first approach for realistic scenarios. Understands space-time trade-off (the dictionary uses extra memory).

Red flag: Only solution is "use a for loop." Does not consider pre-building an index. Tries to sort and binary search without explaining the trade-off vs. a hash-based lookup.

G4. Explain the difference between IEnumerable, ICollection, and IList in .NET. When should you expose each as a return type or parameter?

Expected Answer:

```
IEnumerable<T>          (iteration only)
    |
ICollection<T>          (+ Count, Add, Remove, Contains)
    |
IList<T>                (+ index access, Insert, RemoveAt)
    |
List<T>                 (concrete implementation)
```

Interface	Capabilities	Use as Return Type When
`IEnumerable<T>`	Forward-only iteration; supports LINQ; lazy evaluation	Caller only needs to iterate; you want to support deferred execution
`ICollection<T>`	Count, Add, Remove, Contains	Caller needs to know the size or modify the collection
`IList<T>`	Index access, Insert at position	Caller needs random access by index
`IReadOnlyList<T>`	Index access + Count (no mutation)	Caller needs index access but should not modify

Guidelines:

```
// Accept the most general type as parameter
public decimal CalculateTotal(IEnumerable<LineItem> items)
{
    return items.Sum(i => i.Amount); // Only iterates — IEnumerable is sufficient
}

// Return the most specific read-only type
public IReadOnlyList<TaxReturn> GetReturnsForYear(int year)
{
    // Caller knows the Count and can access by index, but cannot modify
```

```
    return _returns.Where(r => r.TaxYear == year).ToList().AsReadOnly();
}

// Avoid returning List<T> directly — exposes mutation
public List<TaxReturn> GetReturns() // BAD — caller can Add/Remove
public IReadOnlyList<TaxReturn> GetReturns() // GOOD — read-only contract
```

Deferred execution pitfall:

```
// Dangerous — IEnumerable may be a deferred query
public IEnumerable<TaxReturn> GetReturns()
{
    return _db.Query<TaxReturn>("SELECT * FROM TaxReturns");
    // If db connection is closed before enumeration, this throws
}

// Safe — materialize before returning
public IReadOnlyList<TaxReturn> GetReturns()
{
    return _db.Fetch<TaxReturn>("SELECT * FROM TaxReturns");
}
```

What to listen for: Knows the inheritance hierarchy. Follows the "accept general, return specific" principle. Understands deferred execution implications with IEnumerable. Prefers `IReadOnlyList<T>` or `IReadOnlyCollection<T>` for return types to communicate immutability.

↑ Back to Index

PART H — Agile & Process

Gate 1 — Conceptual

H1. What is the difference between Scrum and Kanban? When would you prefer one over the other?

Expected Answer:

Aspect	Scrum	Kanban
Cadence	Fixed sprints (1-4 weeks)	Continuous flow
Roles	Scrum Master, Product Owner, Dev Team	No prescribed roles
Planning	Sprint planning at start of sprint	Just-in-time, replenish when capacity exists
WIP limits	Sprint backlog is the implicit WIP limit	Explicit WIP limits per column

Aspect	Scrum	Kanban
Metrics	Velocity (points per sprint)	Lead time, cycle time, throughput
Change during work	Discouraged mid-sprint	Allowed anytime (within WIP limits)
Ceremonies	Daily standup, sprint planning, review, retro	Daily standup (optional), no required ceremonies

When to prefer Scrum:

- New teams that need structure and predictable delivery cadence.
- Product development with clear sprint goals and release planning.
- Stakeholders need regular demos and forecasting.

When to prefer Kanban:

- Support/maintenance teams with unpredictable incoming work.
- Teams already delivering well that want less ceremony overhead.
- Continuous deployment environments where batching into sprints adds no value.

What to listen for: Does not present one as universally better. Understands that Scrum provides structure through constraints (time-boxing, roles) while Kanban provides structure through WIP limits and flow visualization. Mentions that many teams use a hybrid ("Scrumban").

H2. What is "Definition of Done" and why does it matter?

Expected Answer:

Definition of Done (DoD) is a shared checklist that all work must satisfy before it is considered complete. It ensures consistent quality and prevents "90% done" syndrome.

Example DoD for a development team:

- ☐ Code compiles without warnings
- ☐ Unit tests written and passing (minimum coverage for new code)
- ☐ Integration tests passing
- ☐ Code reviewed and approved by at least one peer
- ☐ No known defects (bugs found during development are fixed)
- ☐ Documentation updated (API docs, architecture decision records)
- ☐ Feature tested in staging environment
- ☐ Meets acceptance criteria from the user story
- ☐ Performance: no regression in response times
- ☐ Security: no new vulnerabilities introduced (static analysis clean)

Why it matters:

1. **Prevents partially done work** — Without DoD, "done" means different things to different people.
2. **Enables predictable velocity** — Points only count when DoD is met, making velocity meaningful.
3. **Quality gate** — Catches issues before they reach production.

4. **Reduces technical debt** — Testing and documentation are not deferred.

What to listen for: Concrete, actionable items in the DoD. Understands that DoD applies to every story, not just some. Mentions that the team owns and evolves the DoD over time. Knows the difference between DoD (applies to all work) and Acceptance Criteria (specific to each story).

Gate 2 — Scenario-Based

H3. Your team has accumulated significant technical debt. The product owner resists dedicating sprint capacity to debt reduction because "there are no new features." How do you make the case?

Expected Answer:

Quantify the impact:

1. **Velocity trend** — Show that velocity is declining sprint over sprint. Technical debt slows down new feature development.
2. **Bug rate** — Track bugs per sprint. High bug rates correlate with unmaintained code.
3. **Lead time** — Measure how long it takes from "start development" to "deployed." If a simple feature takes 2 weeks because of fragile tests and manual workarounds, that is the cost of debt.
4. **Developer onboarding** — New team members take longer to become productive in a debt-heavy codebase.

Framing for the PO:

- "Every sprint, we spend 30% of our capacity working around problems in module X. A 2-sprint investment reduces that to 5%, giving us 25% more capacity for features permanently."
- Use the "interest" metaphor: "We are paying interest on this debt every sprint. The principal is not growing, but the interest is compounding because we keep building on top of it."

Practical approaches:

1. **Boy Scout Rule** — Leave code better than you found it. Budget 10-20% of each sprint for cleanup alongside feature work.
2. **Tech debt backlog** — Maintain a prioritized list. When the PO pushes back on a dedicated sprint, include the highest-impact items in feature stories.
3. **Link debt to features** — "Feature Y requires module X. Module X has 3 critical tech debt items. We can build Y faster if we address them first."
4. **Visualize** — Use a "tech debt wall" or dashboard showing debt items and their impact on delivery.

What to listen for: Does not frame it as "developers vs product owner." Uses data and business impact to make the case. Proposes incremental approaches, not "stop all features for a month." Understands that some debt is acceptable (strategic debt) and the goal is managing it, not eliminating it.

Red flag: "We should just do it without telling the PO." Frames it as a developer-vs-business conflict. Proposes an all-or-nothing approach. Cannot quantify the impact.

H4. How do you estimate work effectively? What are story points and when do they fail?

Expected Answer:

Story points measure relative complexity/effort, not time. A 5-point story is roughly 2.5x the effort of a 2-point story, regardless of who works on it.

Estimation techniques:

Technique	How	Best For
Planning Poker	Team members independently estimate, discuss outliers	Sprint planning, feature stories
T-Shirt Sizing	S/M/L/XL buckets	Roadmap-level estimation, backlog grooming
Reference Story	Compare new work to a well-understood past story	Calibrating team's scale
Timeboxing	"We will spend 2 days max on this"	Spikes, research, proof of concept

When story points fail:

1. **Points become time** — "A 3-point story takes 3 days." This defeats the purpose of abstraction. Points should account for complexity, risk, and uncertainty, not just duration.
2. **Cross-team comparison** — "Team A's velocity is 40, Team B's is 25, so A is faster." Different teams have different baselines; comparing velocities is meaningless.
3. **Gaming** — Inflating estimates to appear productive. Velocity becomes a target rather than a diagnostic tool.
4. **Very uncertain work** — "Investigate why the e-file submission fails 2% of the time." This cannot be estimated because the complexity is unknown. Use a time-boxed spike instead.
5. **Small, uniform tasks** — If all stories are roughly the same size (e.g., CRUD endpoints), counting stories is simpler than pointing them.

What to listen for: Understands that points measure complexity, not time. Mentions the failure modes. Proposes alternatives for situations where points do not work well (spikes, throughput-based forecasting). Does not treat estimation as exact science.

Red flag: "A 1-point story is half a day." Treats velocity as a performance metric for individuals. Cannot explain why cross-team velocity comparison is invalid.

Appendix A — Quick Reference

Pattern Selection Guide

Problem	Pattern
Create objects without specifying exact class	Factory Method
Create families of related objects	Abstract Factory

Problem	Pattern
Construct complex objects step by step	Builder
Ensure only one instance exists	Singleton (prefer DI `AddSingleton`)
Add behavior without modifying existing code	Decorator
Simplify complex subsystem interface	Facade
Convert incompatible interface	Adapter
Control access to an object	Proxy
Swap algorithms at runtime	Strategy
Notify multiple objects of state changes	Observer
Encapsulate requests as objects	Command
Define algorithm skeleton, defer steps to subclasses	Template Method
Pass request along a chain of handlers	Chain of Responsibility
Abstract data access behind collection-like interface	Repository
Track changes across multiple repositories	Unit of Work
Encapsulate business rules as objects	Specification
Reduce direct dependencies between objects	Mediator

.NET Collection Quick Reference

Need	Collection	Lookup	Insert
Key-value pairs	`Dictionary<K,V>`	O(1)	O(1)
Unique elements	`HashSet<T>`	O(1)	O(1)
Ordered list	`List<T>`	O(n) / O(1) index	O(1) amortized
Sorted key-value	`SortedDictionary<K,V>`	O(log n)	O(log n)
Thread-safe key-value	`ConcurrentDictionary<K,V>`	O(1)	O(1)
FIFO queue	`Queue<T>`	N/A	O(1)
LIFO stack	`Stack<T>`	N/A	O(1)

SOLID One-Liner Reference

Principle	One-Liner
Single Responsibility	A class should have one reason to change
Open/Closed	Open for extension, closed for modification

Principle	One-Liner
Liskov Substitution	Subtypes must be usable wherever their base type is expected
Interface Segregation	No client should depend on methods it does not use
Dependency Inversion	Depend on abstractions, not concretions

Cloud Service Decision Matrix

Scenario	Azure	AWS
Host a web API	App Service	Elastic Beanstalk / ECS
Event-driven compute	Azure Functions	Lambda
Message queue	Service Bus Queue	SQS
Pub/sub messaging	Service Bus Topic	SNS + SQS
Object storage	Blob Storage	S3
Relational database	Azure SQL	RDS
Secrets management	Key Vault	Secrets Manager
Identity/Auth	Azure AD (Entra ID)	Cognito / IAM
Container orchestration	AKS	EKS
Serverless containers	Container Apps	Fargate

Appendix B — Gate Assessment Summary

Use this rubric to evaluate where a candidate falls after the interview.

Gate 1 — Junior (0-3 years)

Competency	Pass	Fail
SOLID	Explains each principle; identifies violations in code	Cannot name principles or gives textbook definitions without examples
Patterns	Knows Singleton, Factory, Repository; understands when to use	Confuses patterns or cannot provide code examples
Architecture	Understands layered architecture; knows what MVC is	Cannot explain why layers exist
Testing	Writes basic unit tests; knows what mocking is	Does not test or tests only manually
Git	Commits, branches, merges; creates PRs	Only uses Git GUI with no understanding of operations

Competency	Pass	Fail
Data structures	Knows List vs Dictionary; understands Big-O basics	Uses List for everything; does not know Big-O
Agile	Participates in ceremonies; understands user stories	Cannot explain sprint or standups

Advance to Gate 2 if: Solid conceptual foundation, writes working code, asks good questions when unsure.

Gate 2 — Mid-Level (3-7 years)

Competency	Pass	Fail
SOLID	Applies principles to refactoring decisions; explains trade-offs	Knows principles but cannot apply to real scenarios
Patterns	Uses 5+ patterns appropriately; explains trade-offs	Uses patterns everywhere without justification or cannot combine patterns
Architecture	Explains Clean Architecture vs layered; makes informed choices	"Clean Architecture is always better" without nuance
Testing	Writes integration tests; uses Moq/AutoMock; knows test pyramid	Only writes happy-path tests; does not mock
Cloud	Deploys to cloud; writes Dockerfiles; understands CI/CD	Cannot explain why containers matter; no cloud experience
Git	Resolves merge conflicts; chooses appropriate branching strategy	Panics at merge conflicts; only knows one branching strategy
Data structures	Chooses appropriate collection types; optimizes hot paths	Does not consider performance implications of collection choice
Agile	Estimates work; manages tech debt incrementally	"Tech debt is not my problem"

Advance to Gate 3 if: Makes informed trade-off decisions, designs solutions to real problems, mentors juniors effectively.

Gate 3 — Senior (7-15 years)

Competency	Pass	Fail
SOLID	Teaches principles through code review; knows when to bend rules	Dogmatic application without pragmatism
Patterns	Composes patterns for complex problems; identifies anti-patterns	Over-engineers or cannot simplify
Architecture	Designs systems with appropriate complexity; makes DDD decisions	Every system needs microservices; cannot justify architectural choices

Competency	Pass	Fail
Testing	Designs test strategy; rehabilitates failing test suites; practices TDD	Treats testing as someone else's job
Cloud	Chooses appropriate services and deployment strategies; designs for failure	Cannot explain trade-offs between services
Process	Influences team practices; quantifies technical debt impact	Cannot communicate technical concerns to non-technical stakeholders

Advance to Gate 4 if: Makes system-level decisions, balances business and technical concerns, guides architecture across teams.

Gate 4 — Lead/Architect (15+ years)

Competency	Pass	Fail
System design	Decomposes monoliths; designs for scale, reliability, and maintainability	Designs in a vacuum without considering team/business constraints
Pattern mastery	Selects patterns based on forces and constraints, not preference	"We always use [X pattern] for everything"
Trade-off analysis	Articulates pros, cons, and risks of every decision	Cannot explain why an alternative was rejected
Team leadership	Sets coding standards; designs review processes; manages technical debt strategically	Makes all decisions unilaterally; cannot delegate
Communication	Explains technical decisions to executives; writes ADRs; creates diagrams	Cannot simplify for non-technical audiences

Appendix C — Question Index

#	Topic	Gate	Style	Page
A1	SRP — violation and fix	1	Conceptual	Part A
A2	Open/Closed Principle	1	Conceptual	Part A
A3	LSP, ISP, DIP — brief examples	1	Conceptual	Part A
A4	Refactoring a 2,000-line God class with SOLID	2	Scenario	Part A
A5	DIP overhead debate	2	Scenario	Part A
B1	Singleton — thread-safe in C#	1	Conceptual	Part B
B2	Factory Method vs Abstract Factory	1	Conceptual	Part B
B3	Strategy pattern vs if/else	1	Conceptual	Part B

#	Topic	Gate	Style	Page
B4	Decorator for cross-cutting concerns	2	Scenario	Part B
B5	Builder pattern	2	Conceptual	Part B
B6	Repository pattern	2	Conceptual	Part B
B7	Document processing pipeline design	3	Scenario	Part B
B8	Service Locator anti-pattern migration	3	Scenario	Part B
B9	Notification system design (multi-pattern)	4	Design	Part B
B10	God class decomposition strategy	4	Design	Part B
C1	Layered vs Clean Architecture	1	Conceptual	Part C
C2	DDD: Entities, Value Objects, Aggregates	1	Conceptual	Part C
C3	API versioning + response shaping	3	Scenario	Part C
C4	Mediator pattern (MediatR) benefits and costs	3	Scenario	Part C
C5	Monolith to microservices decomposition	4	Design	Part C
D1	Test doubles: mock, stub, fake, spy	1	Conceptual	Part D
D2	Test Pyramid	1	Conceptual	Part D
D3	Testing async code and controllers	2	Applied	Part D
D4	Rehabilitating a flaky test suite	3	Scenario	Part D
D5	TDD with tax bracket calculator	3	Scenario	Part D
E1	App Service vs Functions vs Containers	2	Applied	Part E
E2	Azure Service Bus vs AWS SQS/SNS	2	Applied	Part E
E3	Multi-stage Dockerfile for .NET	3	Scenario	Part E
E4	Blue-green and canary deployments	3	Scenario	Part E
E5	12-factor app principles	3	Scenario	Part E
F1	Merge vs Rebase	1	Conceptual	Part F
F2	GitFlow, trunk-based, GitHub Flow	1	Conceptual	Part F
F3	Cleaning up messy PR commits	2	Scenario	Part F
F4	Merge conflict resolution and prevention	2	Scenario	Part F
G1	Big-O and .NET collection complexities	1	Conceptual	Part G
G2	Dictionary vs List vs HashSet	1	Conceptual	Part G
G3	Optimizing lookups on 1M records	2	Scenario	Part G
G4	IEnumerable vs ICollection vs IList	2	Applied	Part G

#	Topic	Gate	Style	Page
H1	Scrum vs Kanban	1	Conceptual	Part H
H2	Definition of Done	1	Conceptual	Part H
H3	Making the case for tech debt reduction	2	Scenario	Part H
H4	Story points and estimation failures	2	Scenario	Part H

Total: 42 questions across 8 parts and 4 gates.

↑ Back to Index